BACHELOR OF SCIENCE THESIS

Depth-First Search Tree-Based Routing Protocol for Multi-Hop and Multi-Channel Cognitive Radio Networks

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering

Md. Sharif Mollah
Student ID: 1103078

Adiba Sarker
Student ID: 1102034

UNDER THE SUPERVISION OF

Mamun Ahmed
Department of Computer Science and Engineering
Bangladesh Army International University of Science and Technology

Acknowledgements

We begin by expressing our gratitude to Almighty Allah for granting us the strength, patience, and opportunity to complete this work.

We are sincerely thankful to our supervisor, Mamun Ahmed, for his guidance, constructive feedback, and continued encouragement throughout the development of this thesis. His advice helped us refine both the technical direction of the work and the presentation of the research.

We also acknowledge the support of the faculty members of the Department of Computer Science and Engineering at Bangladesh Army International University of Science and Technology. Finally, we are deeply grateful to our parents, families, teachers, and friends for their patience, motivation, and unwavering support.

Md. Sharif Mollah
Adiba Sarker

Abstract

Static spectrum allocation can leave licensed bands underused even while demand for wireless capacity continues to grow. Cognitive radio networks address this mismatch by allowing secondary users to access temporarily idle spectrum without causing harmful interference to primary users. Routing in a multi-hop, multi-channel cognitive radio network is therefore coupled to channel availability, interface constraints, and the possibility of channel switching.

This thesis presents a multi-edge graph representation in which two neighboring nodes may be connected by several channel-specific edges. A Depth-First Search (DFS)-based routing procedure is used to traverse the graph and form a loop-free routing tree, after which channels are assigned according to whether the network uses a single radio or multiple radios. The implementation is developed in C++ and evaluated through repeated simulations under two conditions: a fixed number of nodes with a varying number of channels, and a fixed number of channels with a varying number of nodes.

The proposed DFS traversal is treated as the primary method. Minimum Spanning Tree (MST) routing and a modified Destination-Sequenced Distance-Vector (DSDV) model are retained only as comparison approaches. The reported experiments indicate that channel switching improves route availability; switching-enabled cases in the source results are described at approximately 95% to nearly 100% routing success. The DFS traversal operates in $O(V + E)$ time for a graph with V vertices and E edges. The study demonstrates how a channel-aware multi-edge representation can support straightforward route discovery while preserving the dynamic structure of a cognitive radio network.

Keywords: cognitive radio network; multi-hop routing; multi-channel network; multi-edge graph; depth-first search; channel assignment; spectrum switching; DSDV; minimum spanning tree.

Contents

List of Figures

List of Tables

List of Abbreviations

ABBREVIATION	MEANING
AODV	Ad hoc On-Demand Distance Vector
BEP	Bit-Error Probability
CR	Cognitive Radio
CRN	Cognitive Radio Network
CSMA/CA	Carrier-Sense Multiple Access with Collision Avoidance
DFS	Depth-First Search
DSA	Dynamic Spectrum Access
DSDV	Destination-Sequenced Distance Vector
DSR	Dynamic Source Routing
FCC	Federal Communications Commission
MAC	Medium Access Control
MEGM	Multi-Edge Graph Model
MHCRN	Multi-Hop Cognitive Radio Network
MRMC	Multi-Radio Multi-Channel
MST	Minimum Spanning Tree
NIC	Network Interface Card
PHY	Physical Layer
PU	Primary User
QoS	Quality of Service
SU	Secondary User
WMN	Wireless Mesh Network

Table A.1. Abbreviations used throughout the thesis.

CHAPTER

01

Introduction

Spectrum scarcity is not only a question of how much spectrum exists, but also how intelligently available channels are discovered, shared, and routed through a dynamic wireless network.

This chapter establishes the problem context, motivates channel-aware routing, and defines the contribution of the DFS-based multi-edge graph model.

1.1 Introduction

The rapid growth of wireless services has increased demand for reliable access to radio-frequency spectrum. At the same time, conventional licensing policies often allocate spectrum statically, even though the degree of utilization varies substantially across time and location.

This mismatch contributes to the widely discussed spectrum-scarcity problem. Merely allocating additional bands is not always practical; a complementary solution is to use existing allocations more efficiently. Cognitive Radio Networks (CRNs) address this challenge through dynamic spectrum access. A cognitive radio can observe its environment, identify a temporarily idle band, and adapt its transmission parameters while protecting licensed primary users.

When several channels are available, route discovery and channel selection become interdependent. A route that is feasible on one channel may disappear when a primary user returns, while another route may become available on a different channel. Consequently, a routing model for a CRN must represent both connectivity and channel availability.

This thesis investigates a graph-based approach for multi-hop and multi-channel cognitive radio networks. The model represents each channel-specific connection as an edge and uses a Depth-First Search (DFS) traversal to form a routing tree. Channel assignment is then performed according to whether channel switching is allowed.

1.2 Conventional Wireless Network

A wireless network connects devices through electromagnetic propagation rather than through guided media such as copper cable or optical fiber. Wireless networks can support mobile access, sensor systems, broadband connectivity, public-safety applications, transportation systems, and emergency communications.

Wireless Mesh Networks (WMNs) are especially relevant to multi-hop communication. Instead of relying exclusively on a small number of central access points, mesh routers can relay packets for one another. This structure extends coverage and provides alternative paths when a direct link is unavailable [1, 2].

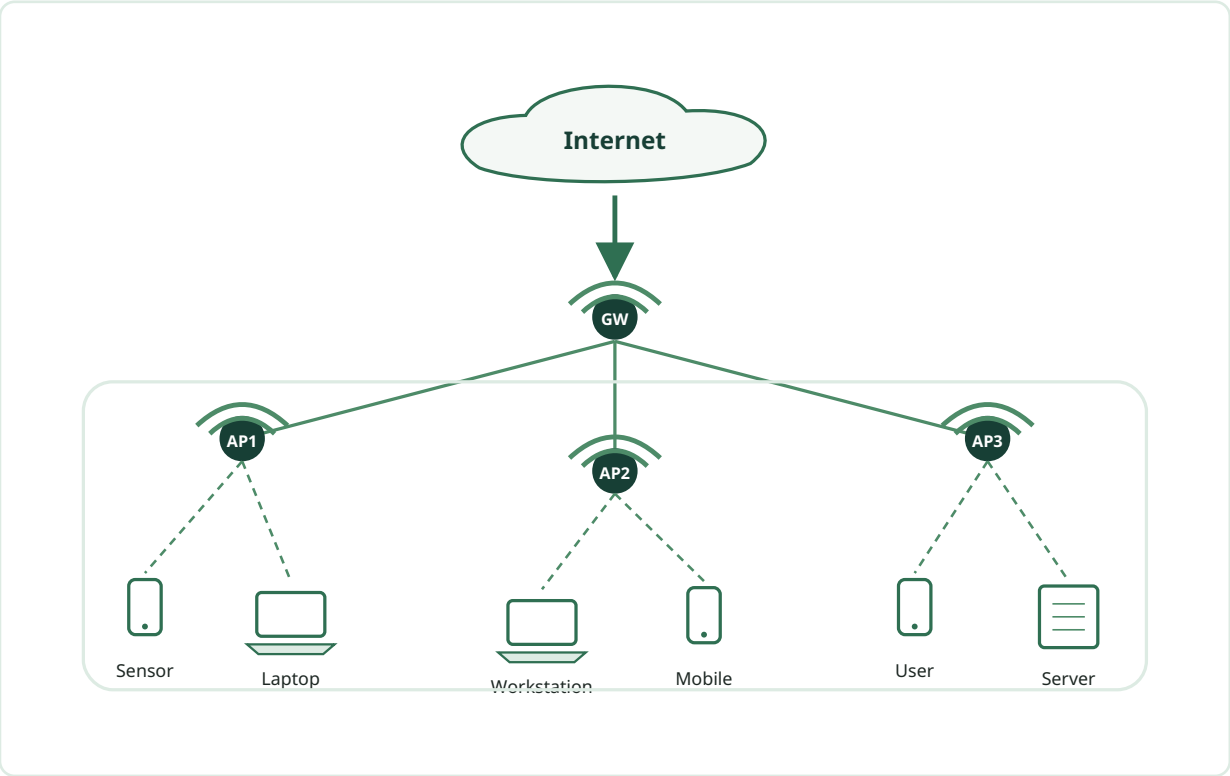


Figure 1.1. *Conventional wireless network architecture showing gateways, access points, and end devices.*

The performance of a conventional wireless network depends on cost, energy consumption, data rate, robustness, throughput, quality of service, and security. These design concerns remain important in a cognitive radio environment, but they are complicated by the fact that channels may appear or disappear as primary-user activity changes.

1.3 Multi-Hop Cognitive Radio Network (MHCRN)

Cognitive radio is an adaptive wireless technology capable of detecting whether a communication channel is occupied and changing its operating parameters accordingly. A Cognitive Radio Network allows secondary users to exploit spectrum opportunities without causing unacceptable interference to primary users.

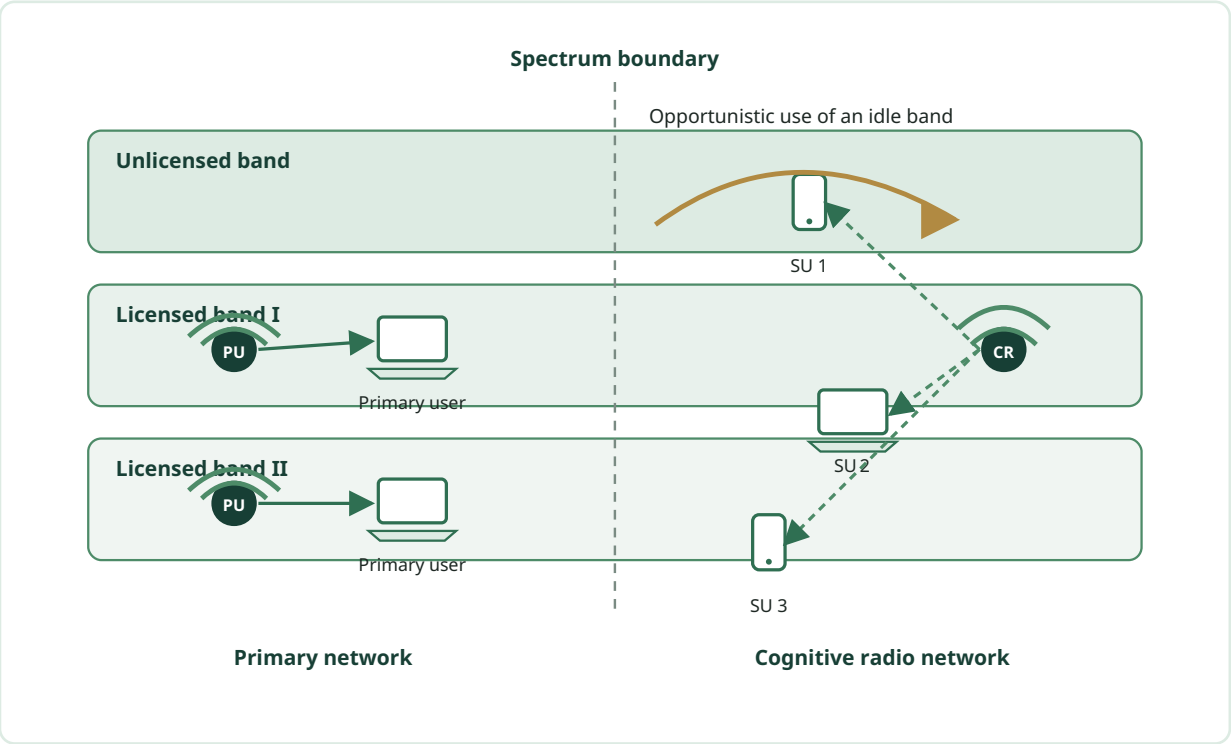


Figure 1.2. *Conceptual structure of primary and cognitive radio networks across licensed and unlicensed bands.*

Two broad forms of cognitive behavior are commonly distinguished. A fully cognitive radio may adapt several parameters, including frequency, modulation, power, and access strategy. A spectrum-sensing cognitive radio focuses primarily on identifying and using available frequency bands. In either case, the radio must vacate or reconfigure its transmission when a primary user reappears.

In a multi-hop network, a destination may lie outside the direct transmission range of the source. Packets are therefore forwarded through one or more intermediate nodes. Relay structures can improve coverage and throughput, particularly at cell edges or in networks where direct links are weak.

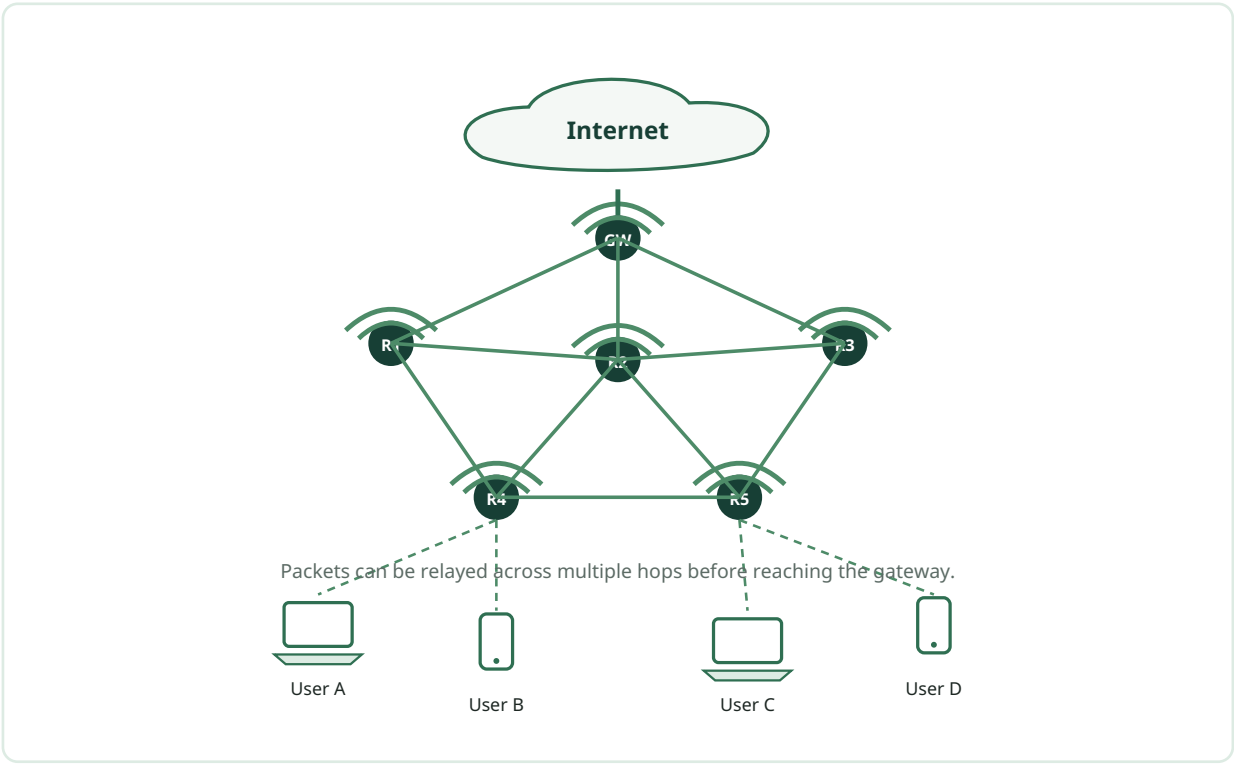


Figure 1.3. *Multi-hop mesh network in which packets may traverse several routers before reaching a gateway.*

A node in an MHCRN may contain one or several radio interfaces. Each interface can be tuned to an available channel. The routing problem must therefore determine not only which neighbor should be selected next, but also which channel should carry each hop.

1.4 Motivation of the Work

Dynamic spectrum access allows secondary users to exploit spectrum holes, but the set of available channels changes with primary-user traffic. A link between two nodes is feasible only when the nodes share at least one available channel at their locations. If that common channel disappears, an established route may fail and a new route must be discovered.

A conventional graph normally represents a pair of neighboring nodes with a single edge. This representation is insufficient when the same pair of nodes can communicate through several channels with different availability or cost. The present work is motivated by the need for a graph model that preserves these parallel channel opportunities without requiring a separate graph layer for every channel.

The proposed solution is a multi-edge graph in which each channel-specific link is represented independently. DFS is used to explore connectivity and construct a traversal tree. This choice provides a simple reachability procedure with $O(V + E)$ traversal complexity. It should be emphasized that DFS does not, by itself, guarantee a minimum-cost or shortest path; its role in this thesis is route discovery and tree formation.

1.5 Importance of Appropriate Channel Selection for Routing

Channel selection directly affects route stability, interference, and service quality. When a primary user becomes active on a channel, a secondary user must avoid harmful interference by moving to another suitable band or pausing transmission. This process is often described as spectrum mobility or spectrum handover.

Candidate channels can be evaluated using availability probability, expected idle duration, signal-to-noise characteristics, interference level, bandwidth, and switching overhead. A routing protocol that ignores these factors may choose a path that is topologically connected but operationally unstable.

Effective communication therefore requires coordination between the routing layer and the channel-assignment process. In a single-radio network, the route may need a common channel across successive links. In a multi-radio network, different input and output channels can be used at an intermediate node, increasing flexibility at the cost of additional coordination.

1.6 Contributions of the Work

The principal objective is to discover a feasible route between a source and a destination separated by multiple hops in a cognitive radio environment. The work makes the following contributions:

- It represents an MHCRN as a multi-edge graph in which parallel edges correspond to distinct available channels.
- It uses a DFS-based procedure to generate a loop-free traversal tree for route discovery.
- It separates the proposed DFS method from the comparison methods: MST and modified DSDV.
- It evaluates single-radio and multi-radio operation, including cases with and without channel switching.
- It implements the model in C++ and performs repeated simulations under fixed-node and fixed-channel conditions.

ELEMENT	ROLE IN THE THESIS
Multi-edge graph	Represents multiple channel-specific links between the same pair of nodes.
DFS traversal	Proposed route-discovery mechanism and topology-formation procedure.
Channel assignment	Selects a common channel for single-radio operation or per-link channels for multi-radio operation.
MST and modified DSDV	Existing or comparison models only; they are not the proposed algorithm.
Simulation	Measures route feasibility under varying node and channel conditions across 100 runs per point.

Table 1.1. Scope and principal contributions of the study.

1.7 Organization of the Thesis

Chapter 2 reviews cognitive radio concepts, channel interference, graph-based routing, spectrum-aware channel assignment, and MST-based routing. Chapter 3 presents the multi-edge system model, the DFS-based topology-formation algorithm, and the channel-assignment procedure. Chapter 4 describes the implementation environment, key C++ data structures, and the simulation setup. Chapter 5 reports and interprets the experimental results and compares the proposed DFS model with MST and modified DSDV. Chapter 6 concludes the thesis and identifies directions for future improvement. The complete source code is included in the Appendix.

CHAPTER

02

Literature Review

Cognitive radio combines spectrum sensing, adaptive decision-making, and graph-based communication. This chapter positions the proposed work within that technical foundation.

> *The literature review distinguishes the proposed DFS traversal from earlier DSDV- and MST-based approaches.*

2.1 Literature Review

The primary motivation for adopting cognitive radio in modern wireless systems is to improve spectrum utilization. The central challenge is to do so without causing harmful interference to licensed users.

Multi-Radio Multi-Channel (MRMC) architectures provide one practical mechanism for increasing flexibility. A node equipped with more than one radio can transmit and receive on different channels, which may reduce contention and improve throughput. However, the benefit depends on how channels are assigned and how routes are selected.

Prior research has examined cognitive radio architectures [3, 6], dynamic spectrum access [4], spectrum management [8], distributed channel assignment [5, 11], interference-aware channel allocation [12–15], graph-based routing [7], and MST-based routing [16]. The present thesis draws on these areas but uses DFS as the proposed graph traversal.

2.2 Cognitive Radio Overview

The demand for wireless spectrum continues to grow while usable radio frequencies remain finite. In a conventional regulatory framework, a band is assigned to a licensed primary user for a defined service or region. Measurements have shown that the utilization of an assigned band can vary, producing temporal or geographic spectrum opportunities.

Dynamic Spectrum Access (DSA) permits a secondary user to access an idle band subject to the requirement that the primary user retains priority. The secondary user must monitor the spectrum, estimate channel conditions, and leave or reconfigure the band when primary-user activity returns.

Spectrum sensing can be implemented through energy detection, matched filtering, cyclostationary feature detection, or cooperative methods. The sensing outcome informs spectrum analysis and channel selection, which in turn influence routing.

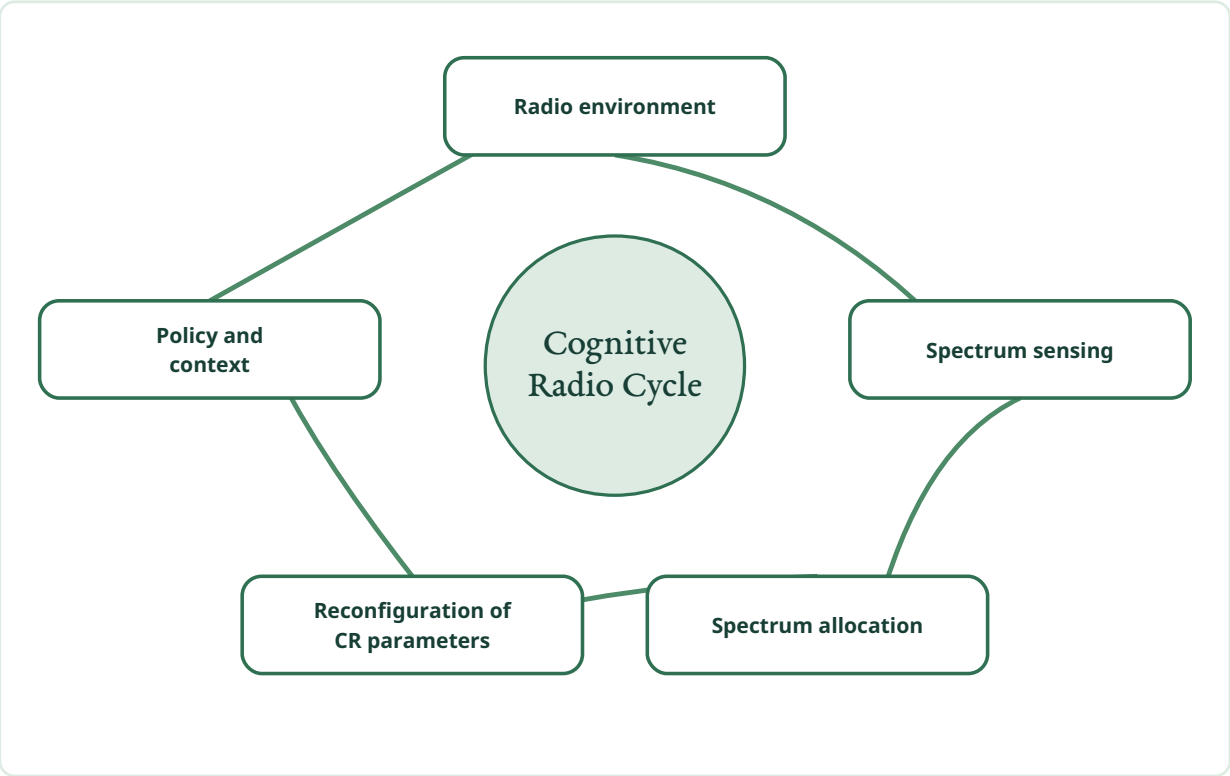


Figure 2.1. *Cognitive radio operating cycle from environment sensing to reconfiguration.*

2.2.1 Characteristics of Cognitive Radio

A cognitive radio system must observe, decide, adapt, and communicate its state. Operational information can include recently detected transmitters, channel occupancy, interference conditions, and the configuration selected by the radio.

The operating environment can change rapidly as traffic, mobility, and primary-user activity vary. A useful CRN must therefore adapt to local conditions and notify neighboring devices when its configuration changes. Because spectrum is a shared and location-dependent resource, resource management is inherently distributed.

These characteristics require a balance between local autonomy and network coordination. A node must react quickly enough to protect the primary user while maintaining a stable route for the secondary communication.

2.2.2 Functions of Cognitive Radio

The core functions of a cognitive radio network are spectrum sensing, spectrum analysis and decision, spectrum sharing, and spectrum mobility. Together, these functions create a closed control loop between the radio environment and the communication strategy.

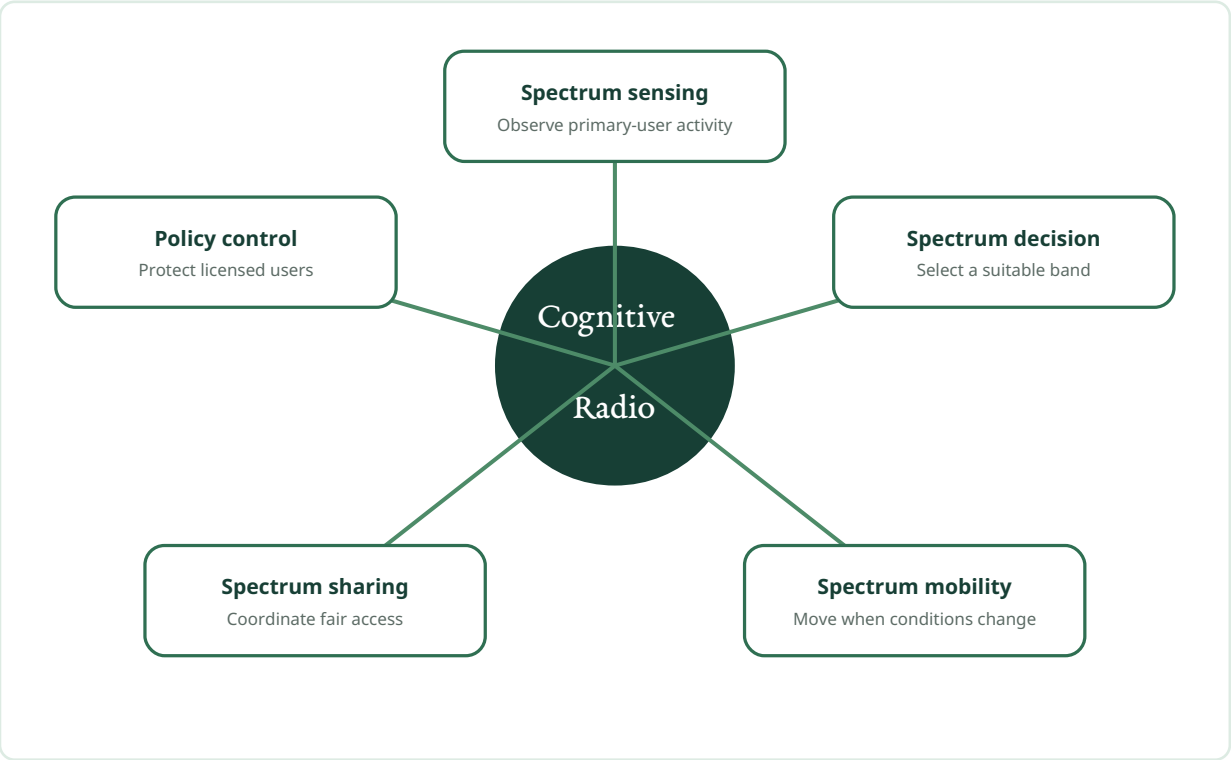


Figure 2.2. *Core functions of a cognitive radio network and their relation to policy control.*

FUNCTION	PURPOSE
Spectrum sensing	Detects primary-user activity and characterizes available bands.
Spectrum decision	Selects a band that satisfies communication and quality-of-service requirements.
Spectrum sharing	Coordinates access among secondary users while limiting interference.
Spectrum mobility	Moves an ongoing communication to another band when conditions deteriorate or a primary user returns.
Policy control	Ensures that adaptation remains consistent with regulatory and network constraints.

Table 2.1. Principal cognitive radio functions.

2.2.3 Cognitive Radio Concept Architecture

A cognitive radio can be viewed as a set of cooperating subsystems rather than a single isolated component. A cognitive unit receives observations from the operating environment and attempts to optimize one or more performance objectives. A policy unit constrains this decision so that regulatory rules and network policies are respected.

The radio platform then executes the selected configuration. A separate sensing subsystem may monitor primary-user activity, channel conditions, and interference. These functions can be distributed across several devices in a CRN, which makes information exchange and synchronized adaptation important.

2.2.4 Communication Channels and Channel Interference

A communication channel is a frequency band used to carry a signal from transmitter to receiver. It is commonly identified by its center frequency, bandwidth, and a channel number. In cognitive radio communication, interference can distort the received signal, raise the bit-error probability, and degrade primary- or secondary-user performance.

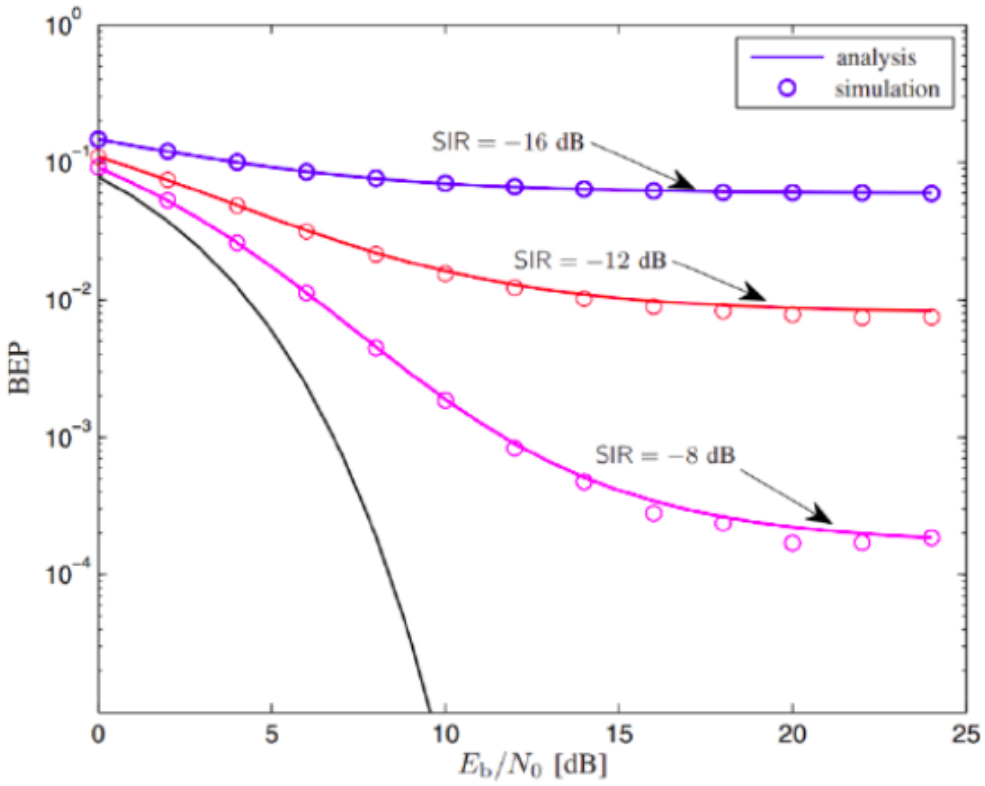


Figure 2.3. *Bit-error probability of a BPSK system in the presence of cognitive network interference, retained from the source document to preserve the original plotted traces.*

The 2.4 GHz IEEE 802.11b channel model illustrates why adjacent-channel interference matters. Neighboring channels overlap substantially, whereas sufficiently separated channels overlap less. The exact set of permitted channels depends on regional regulation, but the conceptual distinction between overlapping and non-overlapping channels remains important.

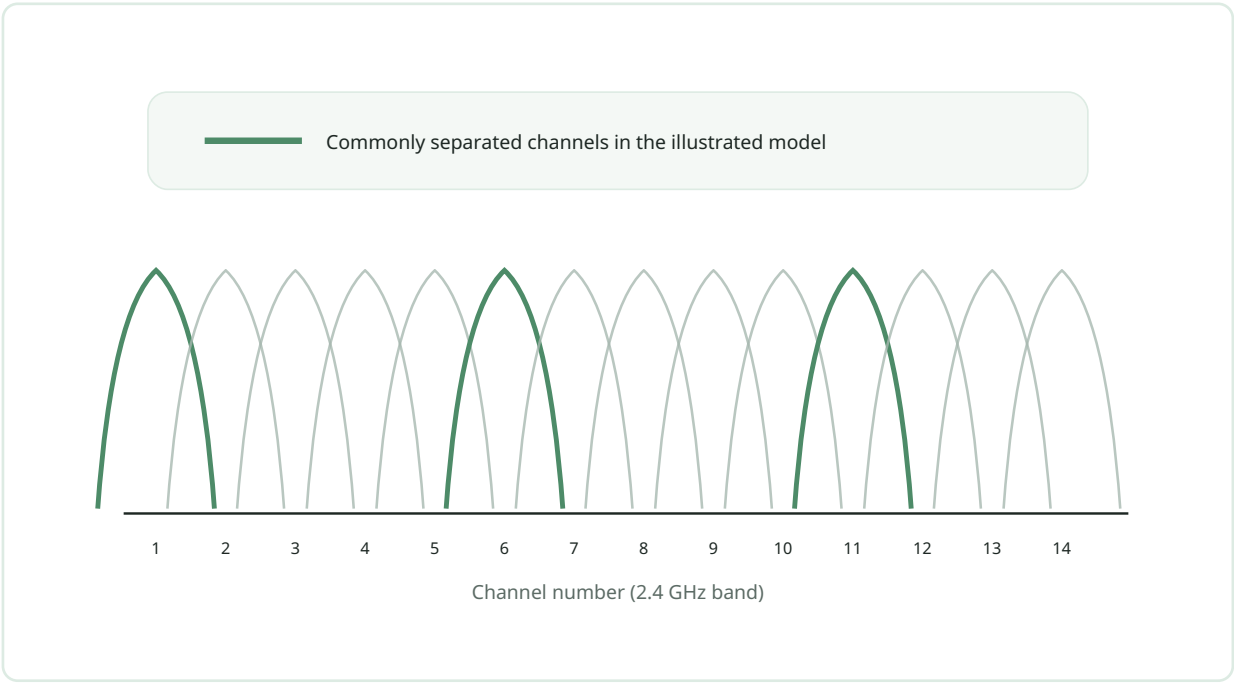


Figure 2.4. *Illustration of overlapping channels in the IEEE 802.11b 2.4 GHz band.*

A receiver can correctly decode a transmission only while the received power remains adequate and interference is limited. The interference range of a transmitter can be larger than its reliable transmission range. A node outside the transmission range may therefore be unable to decode the signal yet still experience interference from it.

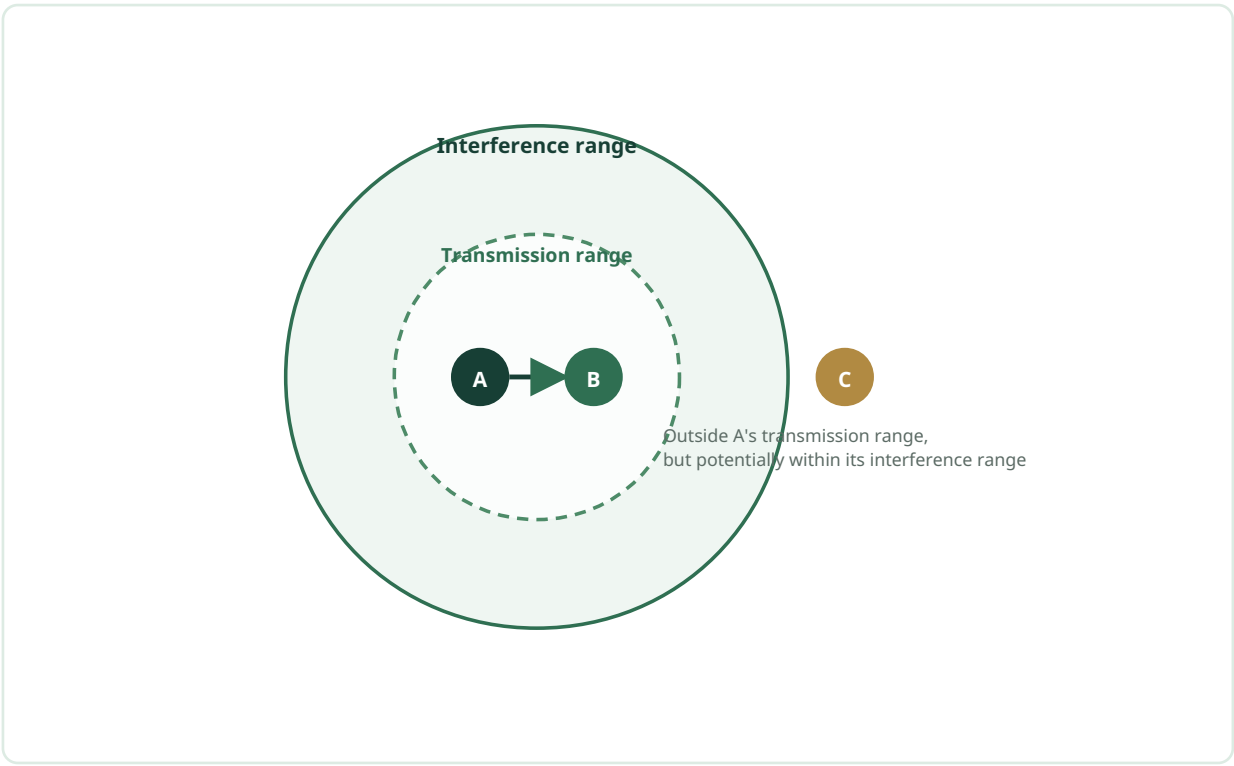


Figure 2.5. *Interference range and transmission range around a transmitting node.*

Two common interference forms are co-channel interference and adjacent-channel interference. Co-channel interference occurs when nearby transmissions use the same channel simultaneously. Adjacent-channel interference occurs when transmissions occupy overlapping bands. Greater spectral overlap generally produces greater adjacent-channel interference.

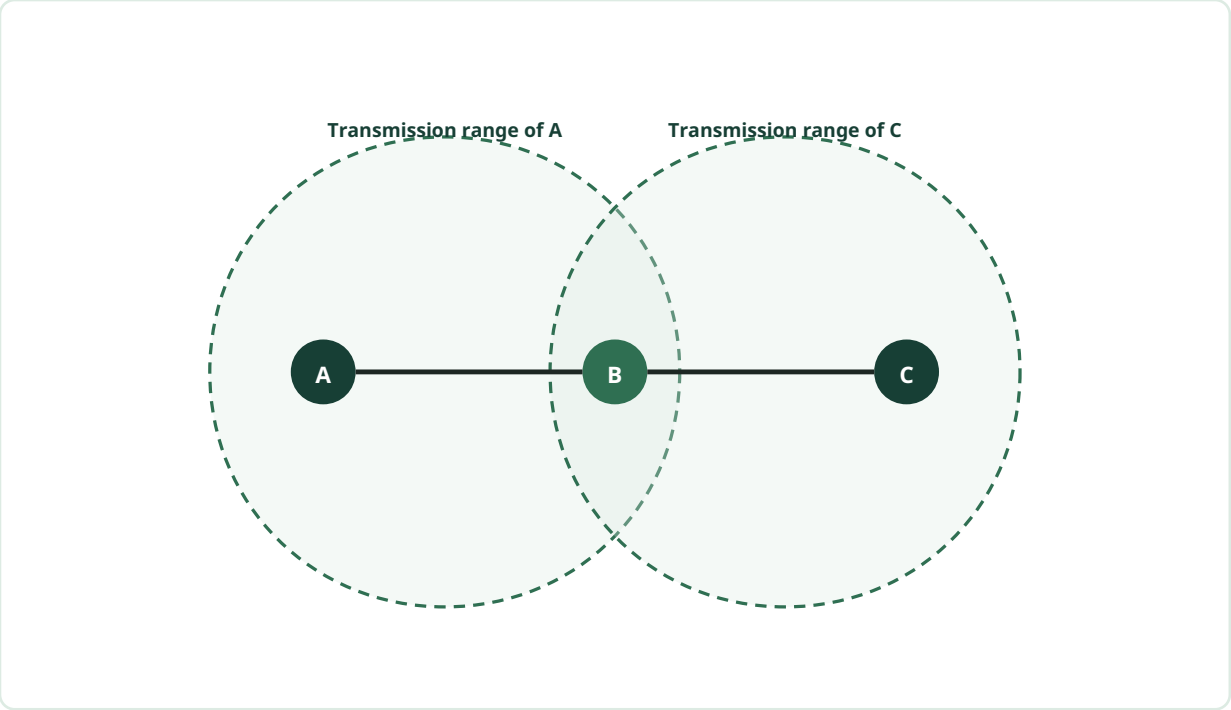


Figure 2.6. *Transmission ranges of nodes A and C overlapping at node B.*

The broadcast nature of wireless communication also creates an overhearing opportunity. A node may receive a packet transmitted to another neighbor if both are within range and use a compatible channel. Overhearing can improve packet delivery in some multicast settings, but uncoordinated simultaneous transmission can also produce collisions. CSMA/CA is commonly used at the MAC layer to reduce such conflicts [10].

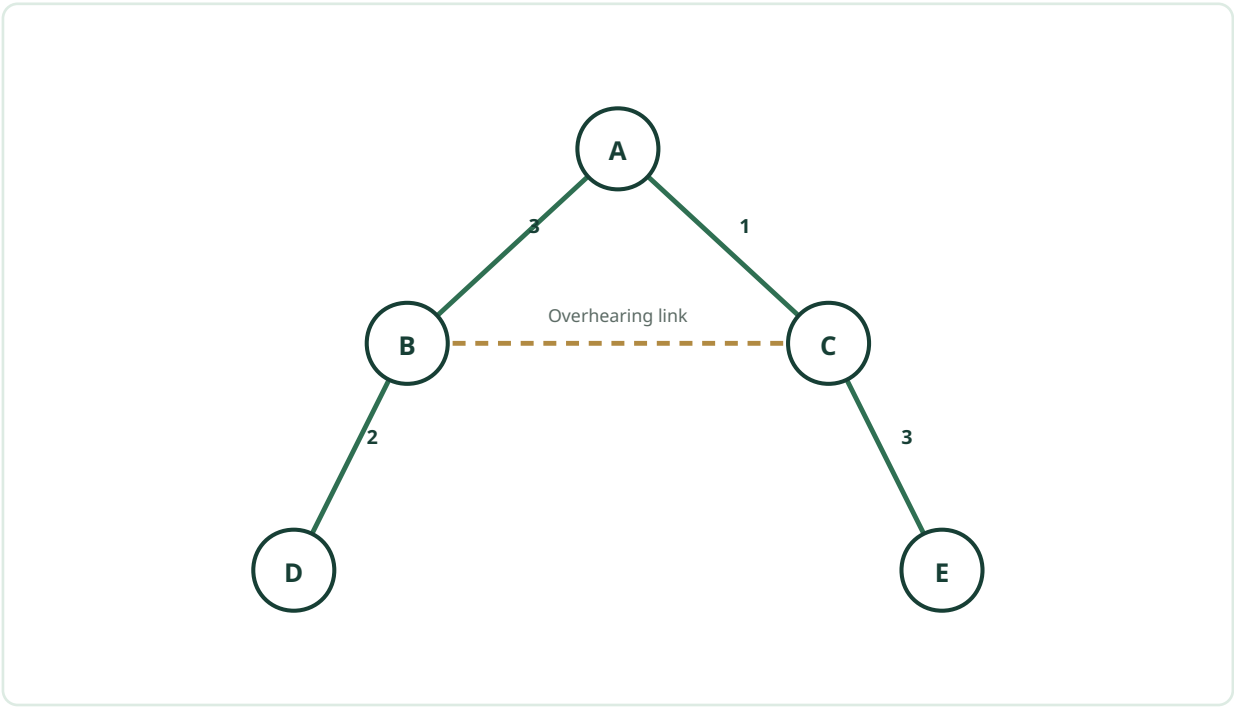


Figure 2.7. *Overhearing effect in a multicast routing tree.*

2.3 Background and Present State of the Problem

In wireless-network literature, the terms *radio* and *interface* are often used interchangeably to describe a network interface connected to an antenna. At a given instant, a radio typically transmits or receives on one selected channel.

Routing must account for transmission range, interference range, neighbor availability, and channel compatibility. These factors are relatively stable in a conventional fixed-channel graph but become dynamic in an MHCRN.

2.3.1 Routing in Graph Representation

Protocols such as AODV, DSDV, and DSR are commonly described through graph models in which vertices represent nodes and edges represent communication links. A link weight can represent distance, delay, interference, energy, or another cost. Traditional routing methods then search the graph for a path that satisfies the chosen objective.

A conventional graph model often assumes that spectrum allocation is fixed and that an available channel remains available. These assumptions do not hold in an MHCRN, where channel availability depends on primary-user activity and can differ from one location to another.

To begin route discovery in a CRN, the network must identify the available channels at each node, determine the common channels for neighboring nodes, and choose among those channels according to an appropriate metric. A multi-edge graph directly represents these alternatives.

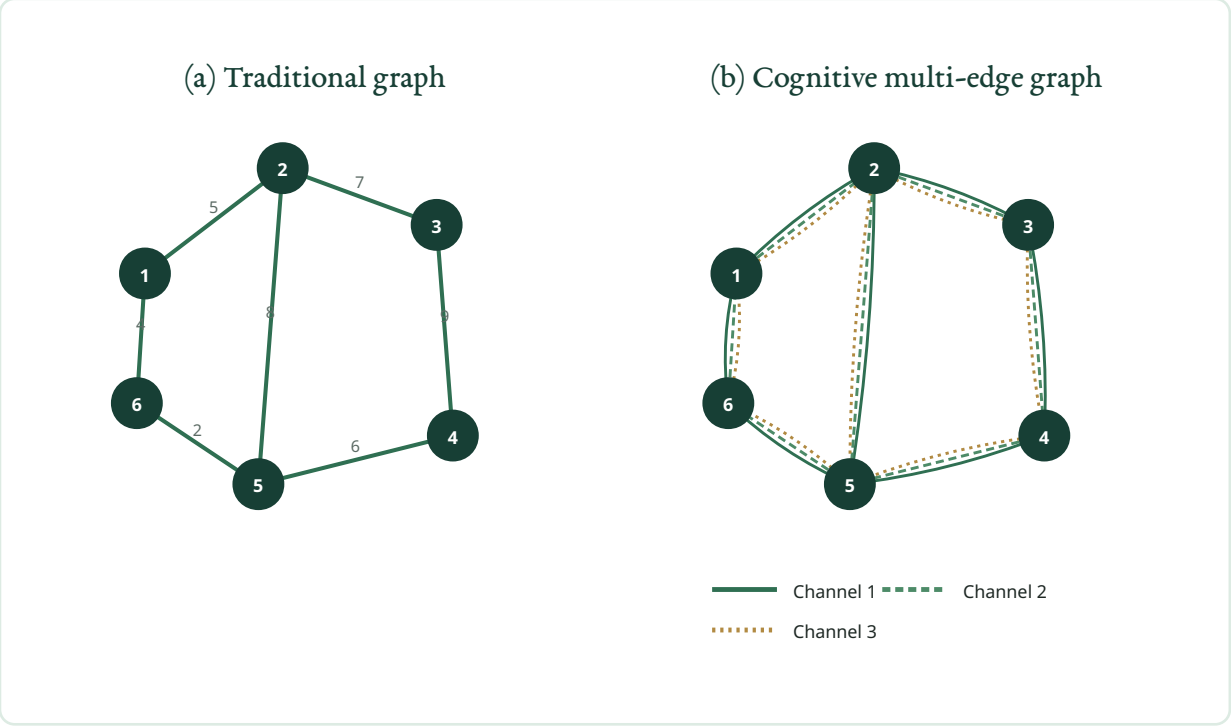


Figure 2.8. *Comparison between a traditional single-edge graph and a cognitive multi-edge graph.*

Kondareddy and Agrawal [7] presented a graph-based routing approach for multi-hop cognitive radio networks. Their model demonstrated that a graph can represent channel-dependent connectivity, while the simulator used a DSDV routing protocol. The present thesis retains modified DSDV only as a comparison model and proposes DFS as the route-discovery traversal.

2.3.2 Spectrum-Aware Dynamic Channel Assignment

Saleem and Bashir [11] proposed a distributed spectrum-sensing and dynamic channel-assignment scheme for MHCRNs. The network first observes primary-user activity. When a channel is available, the scheme evaluates connectivity and interference before assigning a channel to a radio interface.

The objective is to reduce interference to the primary network while improving secondary-user connectivity and transmission performance. The cited work did not explicitly incorporate primary-station occupancy rate into the channel-quality calculation, leaving room for more comprehensive channel metrics.

2.3.3 Channel Assignment

A channel-assignment algorithm determines which channel adjacent nodes should share for communication. Depending on the design objective, the assignment may minimize interference, maximize aggregate throughput, balance load, or reduce switching overhead.

Let $G = (V, E)$ represent a mesh network in which vertices are mesh nodes and edges are direct communication links. Two nodes are neighbors when a feasible wireless link exists between them. In a multi-channel network, a single neighbor relationship may correspond to several channel-specific edges.

The assignment problem is closely coupled to routing. Selecting a channel changes which links are feasible, while selecting a route determines which nodes must share or switch channels.

2.3.4 Minimum Spanning Tree-Based Routing Protocol

An MST connects all vertices of a weighted, undirected graph without cycles while minimizing the total weight of the selected edges. In an MST-based routing protocol, redundant neighbors are removed and the remaining tree is used to forward packets from source to destination.

Sultana and Asaduzzaman [16] reported an MST-based protocol for multi-hop and multi-channel cognitive radio networks. The method seeks to reduce total link cost while preserving connectivity. In this thesis, MST is not the proposed method; it is retained as an existing comparison approach against which the DFS-based model is discussed.

2.4 Chapter Summary

This chapter reviewed cognitive radio terminology, dynamic spectrum access, spectrum sensing, spectrum management, channel interference, graph-based routing, distributed channel assignment, and MST-based routing. The review establishes the need for a representation that preserves several channel alternatives between the same nodes. The next chapter develops that multi-edge representation and defines the proposed DFS-based routing procedure.

CHAPTER

03

Methodology

The methodology models channel opportunities as parallel edges, explores the graph with a stack-based DFS traversal, and assigns channels according to radio constraints.

> *Proposed model: DFS-based routing with channel assignment. Comparison models: MST and modified DSDV.*

Methodological consistency. Throughout this thesis, DFS is the proposed routing and topology-formation method. MST and modified DSDV are discussed only as existing or comparison methods. No breadth-first search procedure is used in the proposed implementation.

3.1 Representation of the System Model

A cognitive radio transceiver observes channel occupancy and adapts its operating frequency to an available band. The system model must therefore represent a network whose links depend on both physical proximity and channel availability.

The model contains a set of nodes located within a communication region. A pair of nodes can communicate when they are within transmission range and share at least one available channel. The number of nodes and the available channel set may vary between simulations.

Each node stores a neighbor list, a set of channels associated with each neighbor, a link distance or cost, and a radio-interface condition. The source and destination are selected from this graph. The routing procedure determines whether a feasible multi-hop path exists and records the edges selected during traversal.

SYMBOL	MEANING OR ASSUMPTION
G = (V, E)	Multi-edge graph representing the cognitive radio network.
V	Set of cognitive radio nodes.
E	Set of channel-specific edges between neighboring nodes.
C	Set of possible communication channels.
w(e)	Weight associated with an edge, such as distance or another link cost.
Single-radio case	A route must preserve channel compatibility without per-hop switching.
Multi-radio case	Input and output channels at an intermediate node may differ.
Simulation assumption	A node is assigned a limited number of neighbors to control graph density.

Table 3.1. Symbols and principal assumptions of the system model.

3.2 Multi-Edge Graph Model

In a conventional graph, two vertices are typically connected by at most one edge. In a multi-edge graph, the same pair of vertices can be connected by several edges. This property is useful for an MHCRN because each edge can represent a different communication channel.

The model captures three elements that a simple graph can obscure: the set of available channels between two nodes, the alternative routes produced by those channels, and the channel-specific neighborhood of each node. The structure remains compact because it does not require a fully separate graph layer for every channel.

3.2.1 Graphical Representation of a CRN

Routing in an MHCRN depends on the available channels reported by the physical layer. The routing layer also needs to know how many radio interfaces are available at each node. An intermediate node with multiple interfaces may receive on one channel and transmit on another, which can reduce contention and increase the probability of maintaining a route.

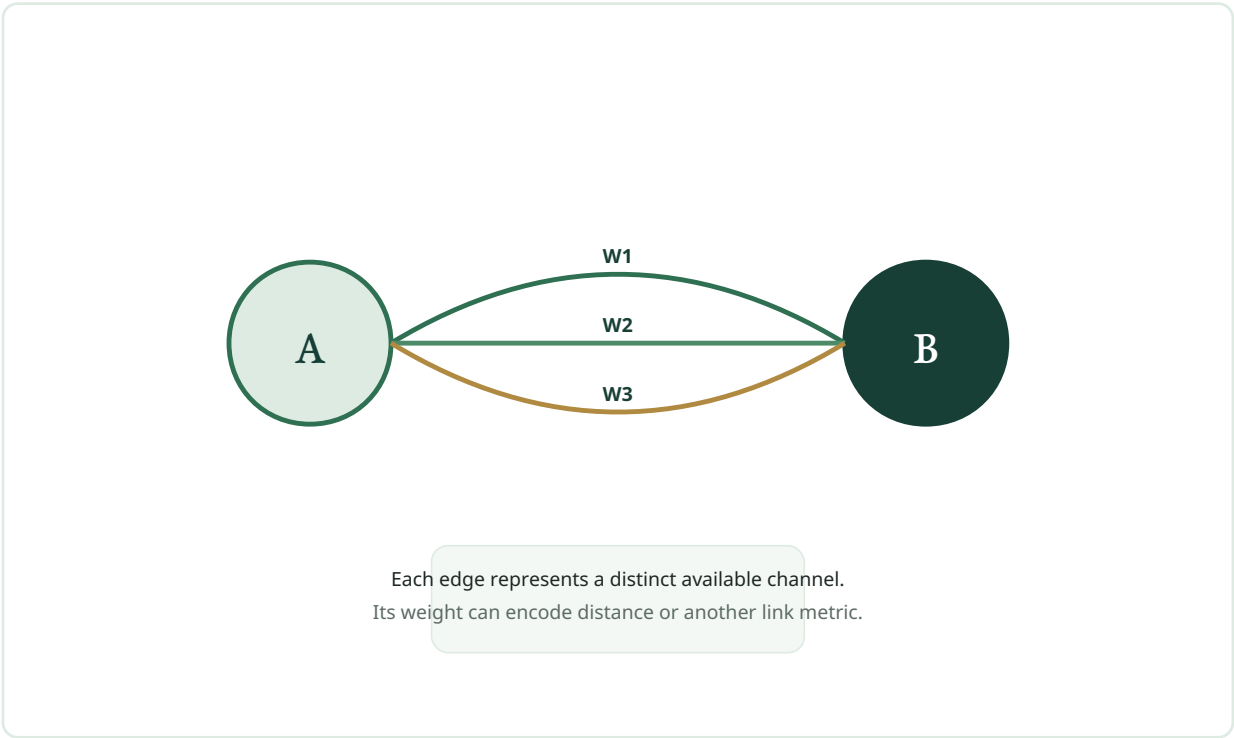


Figure 3.1. *Nodes A and B connected by three channel-specific edges with weights W1, W2, and W3.*

Let G be a multi-edge graph, V the set of nodes, and C the set of possible channels. If nodes A and B share three available channels, they are represented by three parallel edges. The weight of an edge may encode distance or a composite metric such as interference, energy, or expected availability.

In the implementation described in this thesis, distance is stored as a link cost, while channel identity is stored separately. The DFS traversal tests reachability and records selected edges. It does not prioritize edges by cost and therefore should not be described as a shortest-path algorithm.

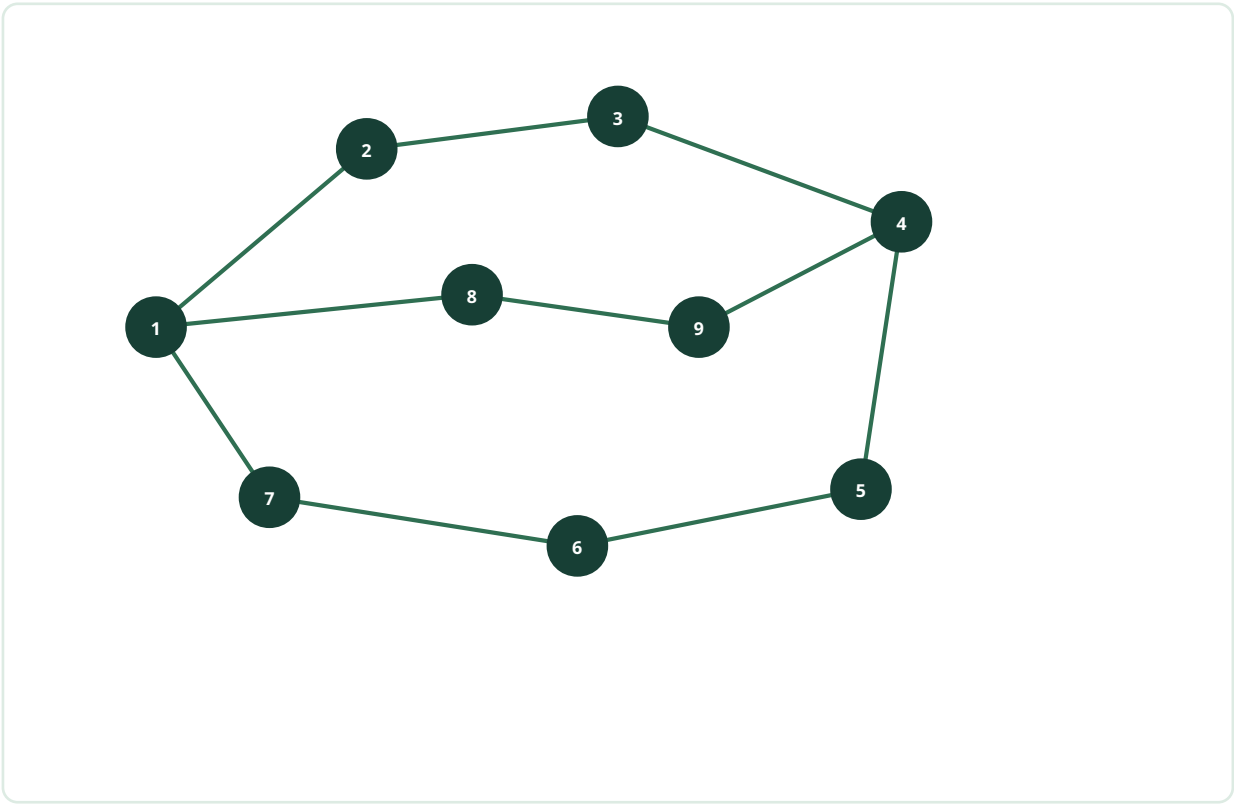


Figure 3.2. *Traditional single-edge representation of the nine-node sample network.*

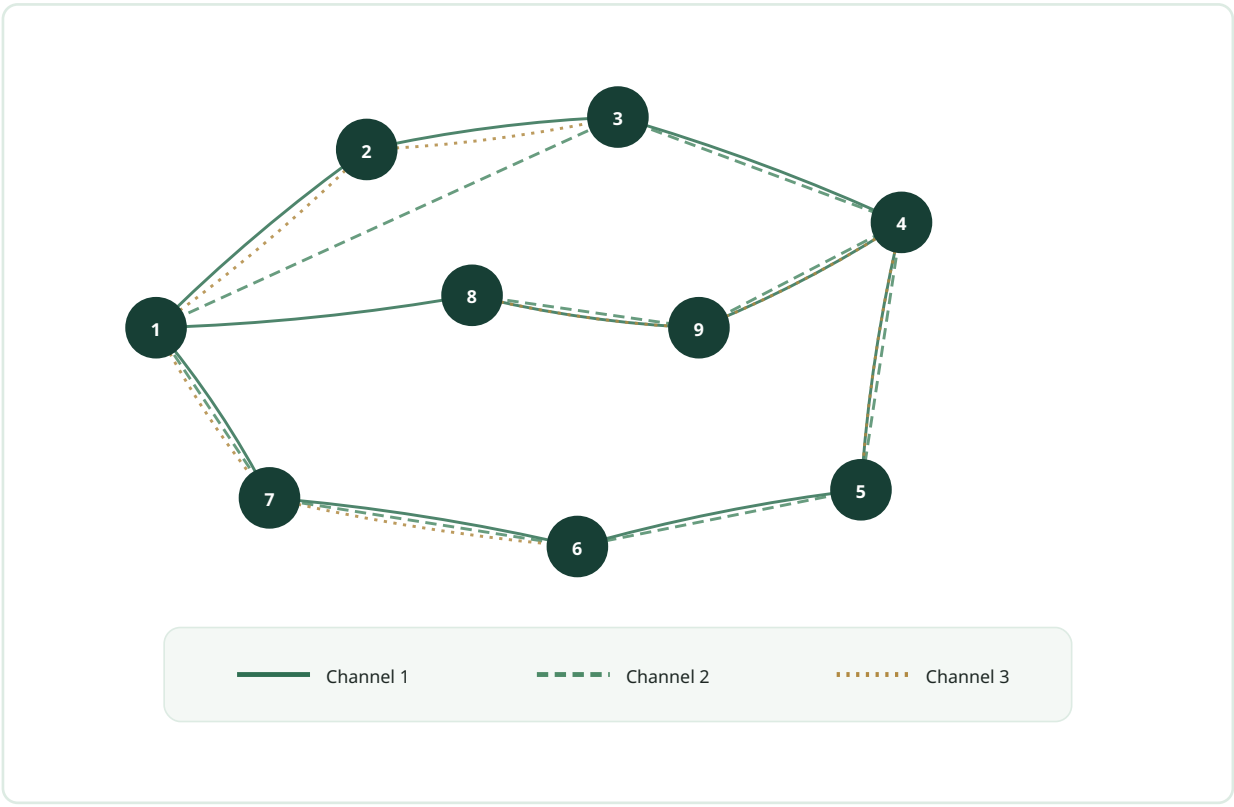


Figure 3.3. *Cognitive multi-edge representation of the sample network, with separate styles for three channels.*

3.2.2 Topology Formation

Topology formation identifies a loop-free set of edges that can be used for routing. The proposed method performs DFS from the source node. A stack stores candidate outgoing links, and a visited array prevents a node from being added more than once.

When a link is removed from the stack, the destination node of that link is examined. If the node has not been visited, the edge is added to the traversal tree, the node is marked as visited, and its feasible outgoing links are pushed onto the stack. This process continues until the stack is empty.

The resulting structure is a DFS tree or forest over the reachable portion of the graph. The method is scalable in the sense that each vertex and edge is processed a bounded number of times, producing $O(V + E)$ traversal complexity.

3.2.3 Weight Assignment and Channel Switching

Graph-based protocols can use edge weights to represent distance, interference, energy, delay, or another objective. A cost-aware routing algorithm may select a route by minimizing the sum of these weights. In the present implementation, however, DFS explores links in stack order; the stored cost is accumulated but not used to guarantee an optimal path.

Channel switching determines which edges are eligible during traversal. When switching is allowed, the algorithm may follow a valid outgoing link even if its channel differs from the channel used on the preceding hop. When switching is not allowed, only links that support the selected channel are pushed onto the stack.

3.2.3.1 INTERFACE CONSTRAINT

A node can contain one or more radio interfaces. Each interface can be tuned to a required channel, but switching introduces synchronization and control overhead. In a cognitive network, channels can be widely separated in frequency, making rapid packet-by-packet switching impractical in some settings.

The model therefore evaluates two cases. In the single-radio case, the route is constrained by a common channel. In the multi-radio case, channels may change between incoming and outgoing links. This distinction is central to the experimental comparison.

3.3 Proposed Algorithm

The route-discovery process contains three stages: graph generation, DFS-based topology formation, and channel assignment. The pseudocode below is aligned with the stack-based C++ implementation in the Appendix.

3.3.1 Algorithm 1: Generate Graph

Algorithm 1 - Multi-edge graph generation

```
Input: total number of nodes N, maximum channel count K
Output: multi-edge graph G = (V, E)

1. Create one vertex for each node i in {1, ..., N}.
2. For every node i, choose a limited set of neighboring nodes.
3. For each neighbor j:
      a. Generate a link distance or cost.
      b. Generate one or more channel identifiers.
      c. Store the symmetric neighbor, distance, and channel data.
4. Return G = (V, E).
```

The graph generator limits each node to a small number of neighbors. This reduces graph density and reflects the fact that a wireless node is unlikely to connect directly to every other node in the network.

3.3.2 Algorithm 2: Topology Formation Using DFS

Algorithm 2 - DFS-based topology formation

```
Input: multi-edge graph G = (V, E), source s,
       destination d, selected channel c,
       switching flag switchAllowed
Output: DFS routing tree T and reachability result

1. Mark all vertices unvisited; mark s visited.
2. Create an empty stack S and an empty edge list T.
3. Push every feasible outgoing edge of s onto S.
4. While S is not empty:
      a. Pop edge (u, v, channel, cost).
      b. If v is unvisited:
           i. Add (u, v, channel) to T.
          ii. Mark v visited.
         iii. If v = d, record that the destination is reachable.
          iv. Push every feasible outgoing edge of v onto S.
5. Return T and the reachability result.
```

An edge is feasible when its channel satisfies the switching condition. With switching enabled, the first available channel associated with the link can be used. Without switching, the link must support the selected channel.

3.3.3 Algorithm 3: Channel Assignment

Algorithm 3 - Channel assignment for the selected DFS tree

```
Input: DFS tree T, channel sets on its edges,
       radio mode (single or multiple)
Output: channel assigned to each selected hop

1. If the network is single-radio:
      a. Identify a channel that is common to the required route edges.
      b. Reject the route if no such common channel exists.
2. If the network is multi-radio:
      a. For each selected edge, choose an available channel.
      b. Permit adjacent hops to use different channels.
3. Record the assigned channel for every selected edge.
4. Return the channel-labeled routing tree.
```

This separation makes the methodological structure explicit: DFS forms the routing tree, while channel assignment determines whether that tree is feasible under the selected radio mode.

3.4 Pictorial Representation of the Sample Network

The following sequence recreates the sample traversal as a consistent editorial figure. The active node is highlighted in gold. Steps 9 to 11 illustrate backtracking, and the final panel shows the selected DFS tree in dark green.

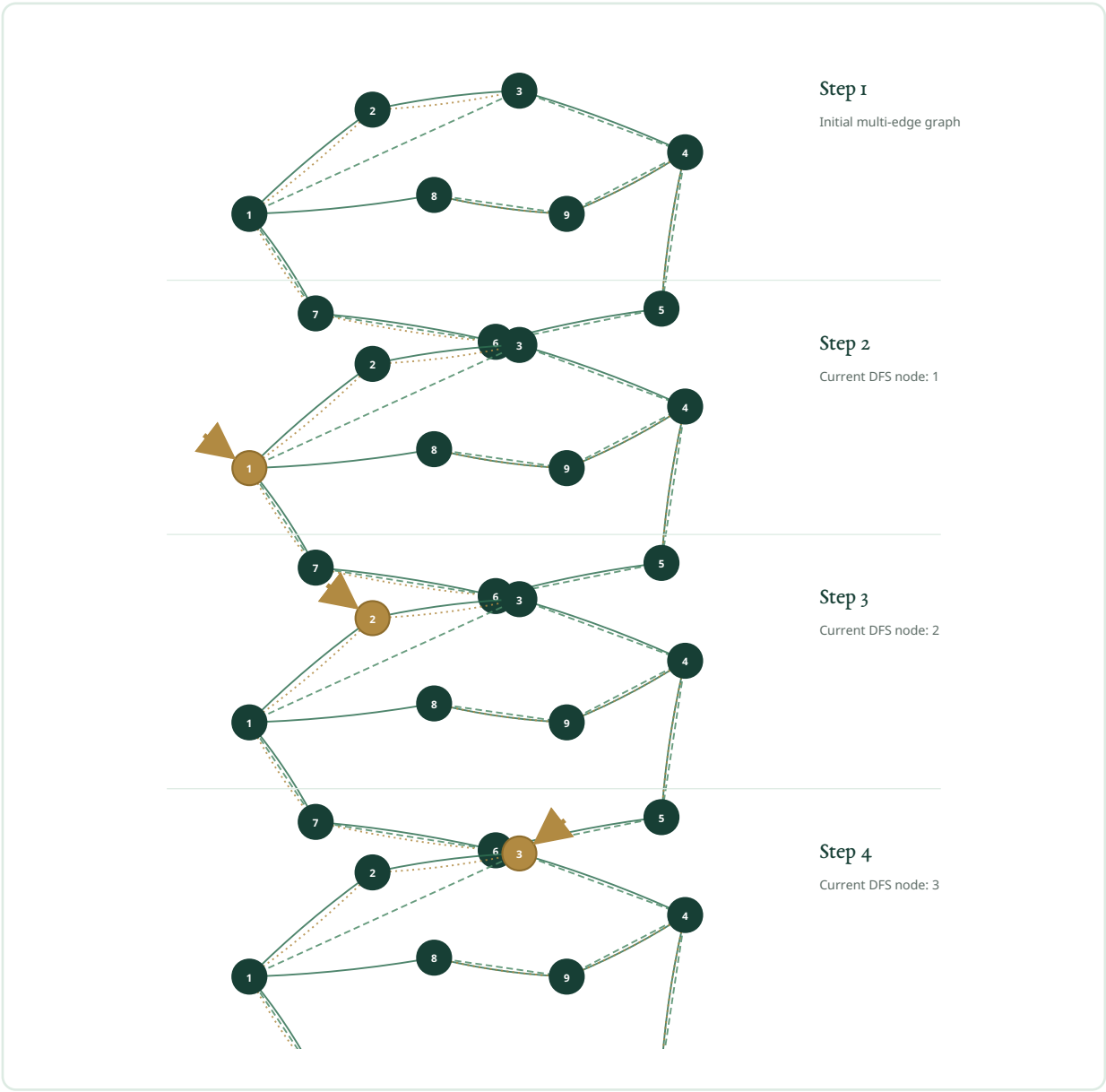


Figure 3.4(a). *DFS traversal of the sample multi-edge network, Steps 1-4.*

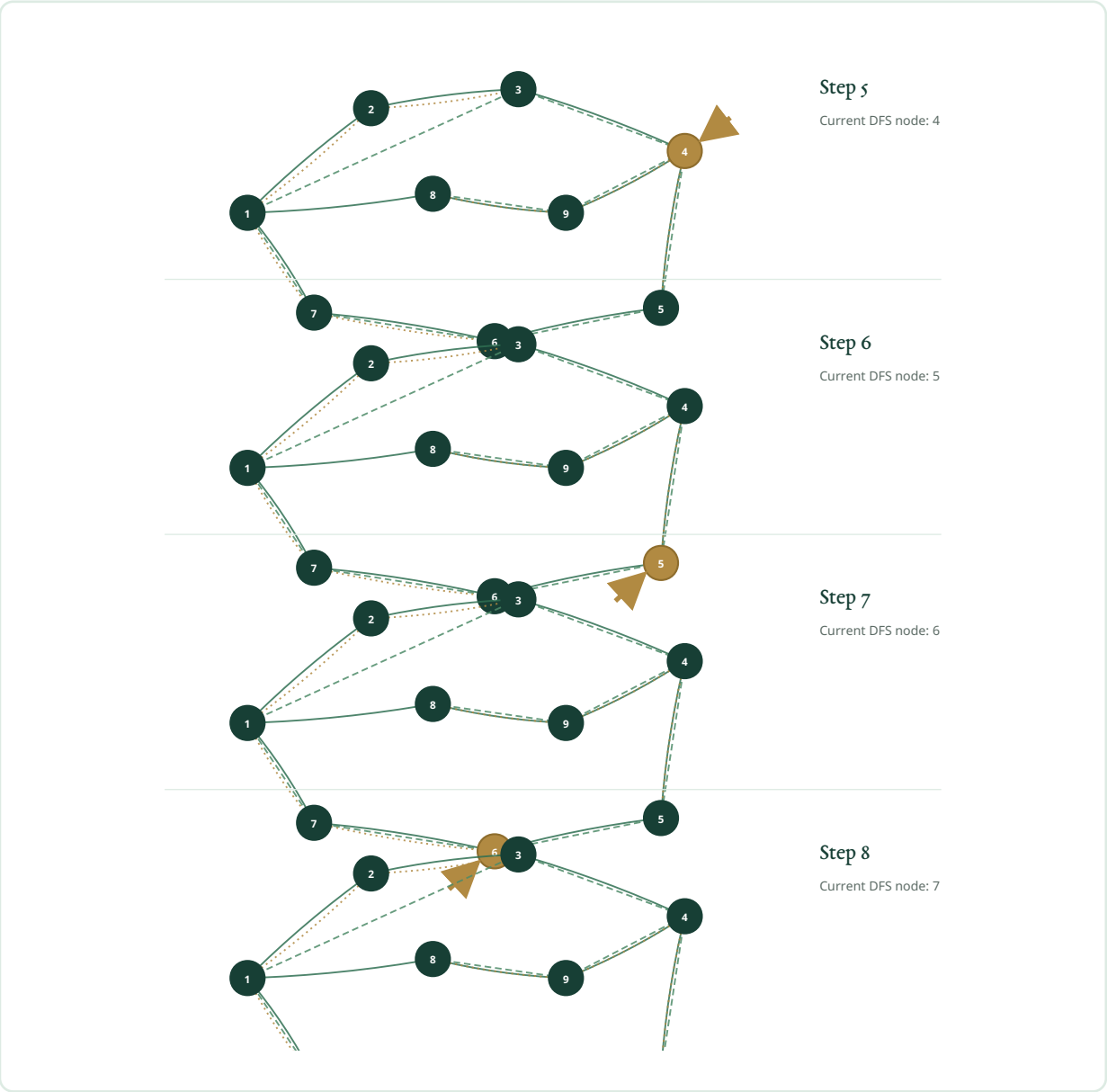


Figure 3.4(b). *DFS traversal of the sample multi-edge network, Steps 5–8.*

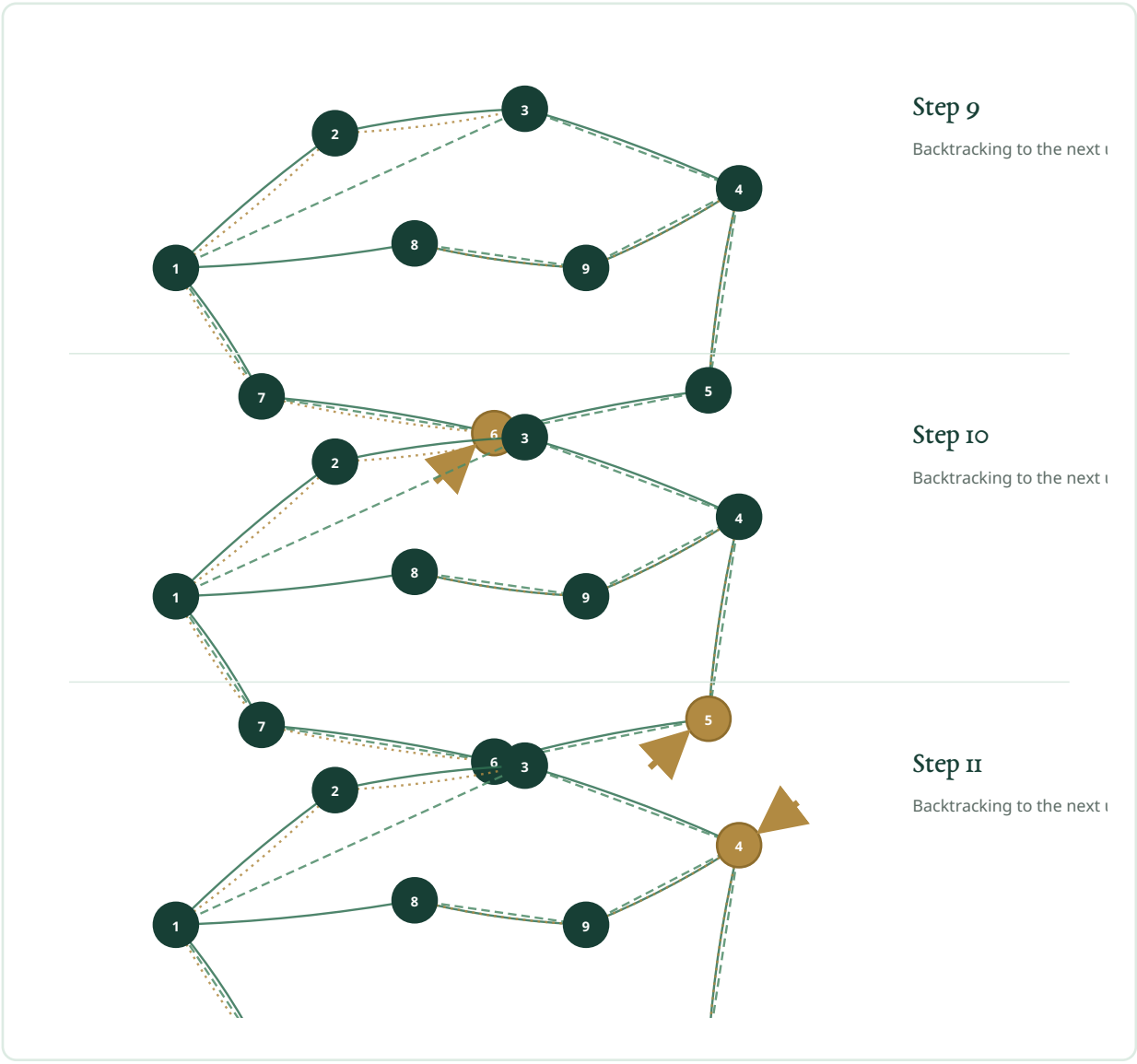


Figure 3.4(c). *DFS traversal and backtracking, Steps 9-11.*

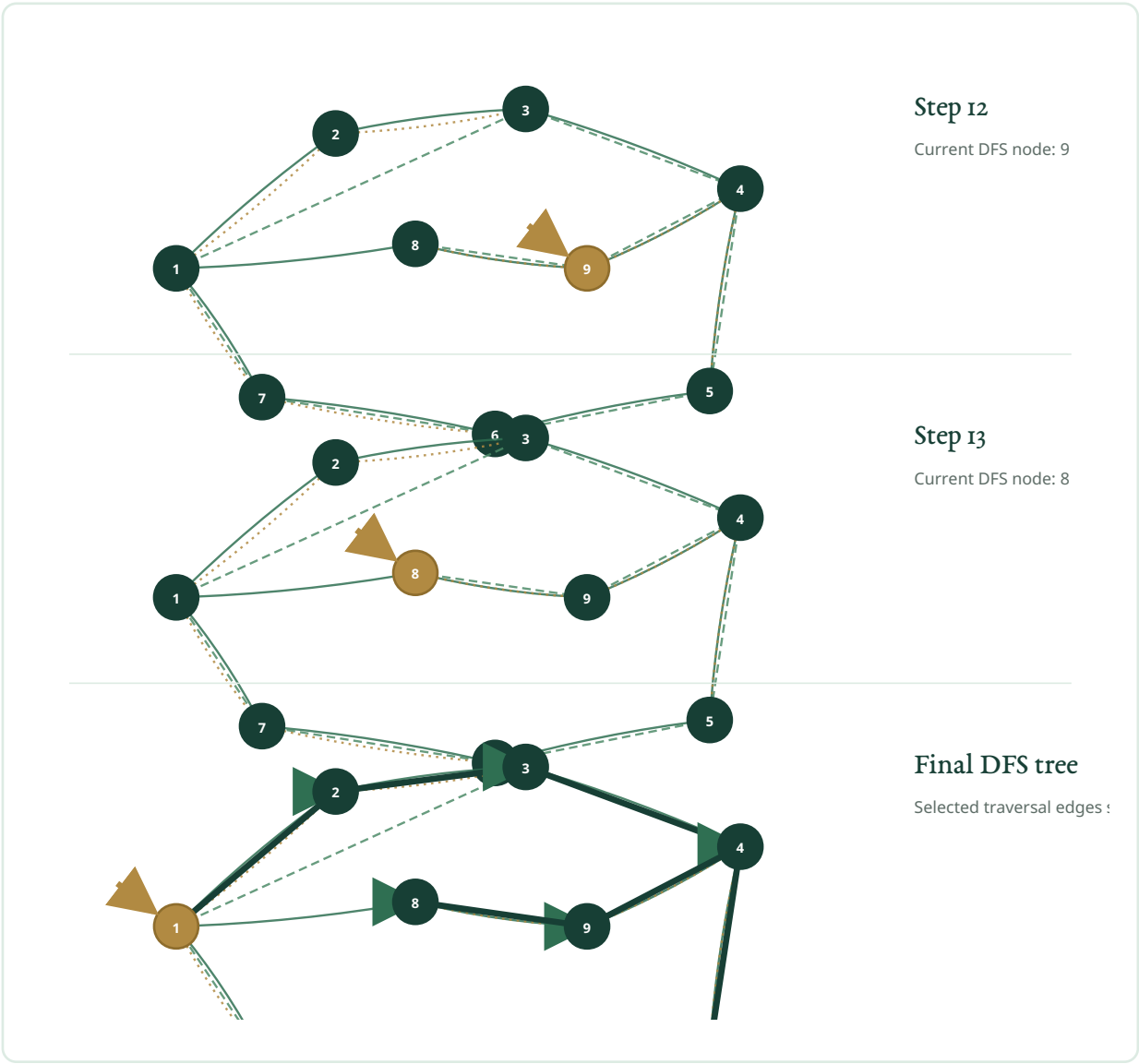


Figure 3.4(d). *DFS traversal, Steps 12-14, ending with the selected routing tree.*

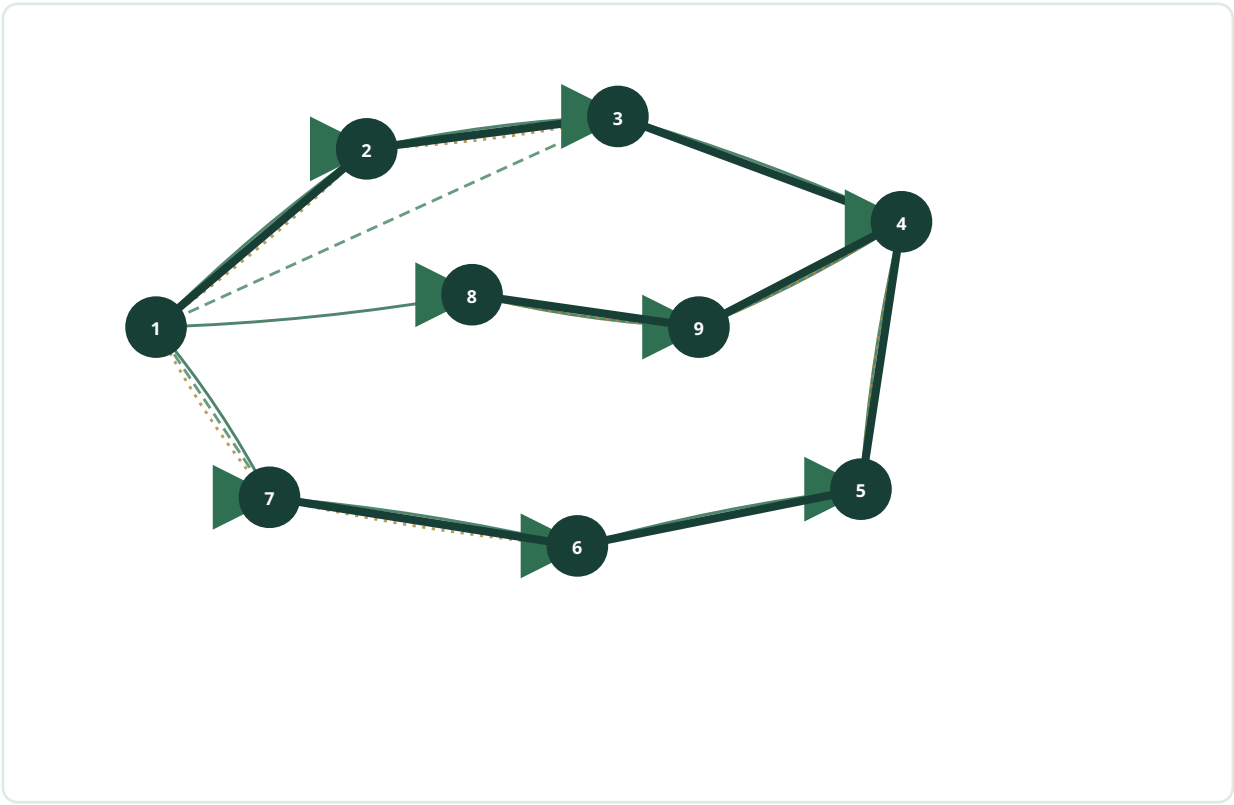


Figure 3.5. *Final DFS routing tree over the sample cognitive radio network.*

3.5 Channel Assignment

After the topology has been formed, channels are assigned to the selected edges. The feasibility of the resulting route depends on whether the radio is allowed to switch channels between hops.

3.5.1 Single-Radio Network

For the source example described in the original model, a packet begins at node 1 and follows the DFS traversal through successive nodes. Without switching, each selected link must support the same channel. The reported traversal sequence is:

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Backtracking then explores the remaining branch:

7 -> 6 -> 5 -> 4 -> 9 -> 8

The exact route depends on stack insertion order and the generated graph. The essential restriction is that the selected channel must remain available along the required links.

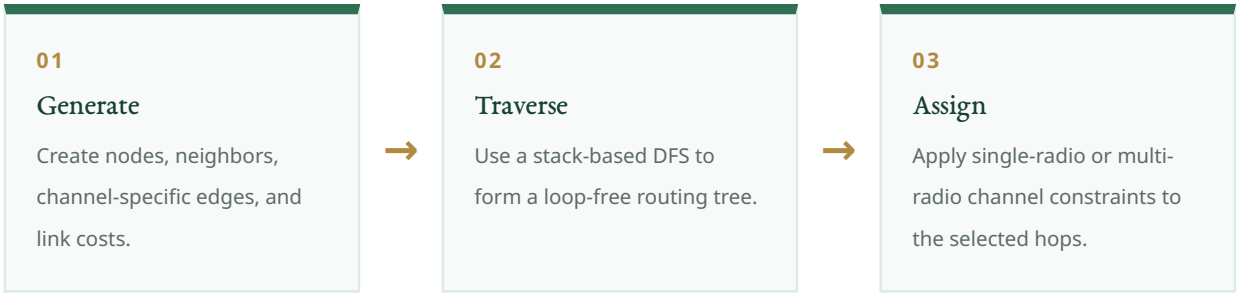
3.5.2 Multi-Radio Network

In a multi-radio network, an intermediate node can receive on one channel and transmit on another. The source document illustrates this flexibility by allowing a route from node 1 toward node 7 through a sequence of available channels, followed by a continuation from node 7 toward node 8 using another channel sequence.

The purpose of the example is not to claim a unique minimum-cost route, but to show that per-hop channel selection makes a route feasible even when no single channel is common to the entire path. This is why channel switching generally produces a higher route-success rate than the single-radio, no-switching case.

3.6 Chapter Summary

This chapter defined the multi-edge graph model, clarified the interface constraint, and presented three coordinated procedures: graph generation, DFS-based topology formation, and channel assignment. DFS is the proposed route-discovery method; MST and modified DSDV remain comparison models. The next chapter describes how these procedures are implemented in C++.



CHAPTER

04

Implementation Details

The implementation stores node, neighbor, channel, and distance data in C++ containers and performs route discovery with an explicit stack.

> *The source code confirms the proposed traversal: the solve() function uses a stack and implements Depth-First Search.*

The proposed model is implemented as a software simulator. This chapter describes the execution environment, the principal data structures, the existing comparison logic, the DFS implementation, and the simulation procedure.

CATEGORY	SPECIFICATION
Computer	Personal computer
Processor	Intel Core i5, 8th generation, 1.60 GHz, 8 logical processors
Graphics	Intel UHD Graphics 620 (Kaby Lake GT2)
Operating system	Ubuntu 18.04.2 LTS
Platform	Linux
Programming language	C++
Development environment	Code::Blocks 16.01

Table 4.1. Hardware and software environment used by the original implementation.

4.1 Functions and Data Types

The implementation uses standard C++ containers and utility functions to store graph information and control traversal.

4.1.1 Stack

A stack follows the Last-In, First-Out (LIFO) principle. Elements are added with `push()` and removed with `pop()`. The DFS procedure stores candidate edges in a stack. The most recently pushed edge is explored first, which produces depth-oriented traversal and backtracking.

4.1.2 Vectors

A C++ `vector` is a dynamic sequence container whose size can change as elements are inserted or removed. The implementation uses vectors to store each node's neighbor list, distance list, and channel lists.

4.1.3 `size()`

The `size()` member function returns the current number of elements in a container. It is used to iterate through the populated neighbor and channel lists. The value is distinct from the container's allocated capacity.

4.1.4 `empty()`

The `empty()` function returns `true` when a container contains no elements. During DFS, the traversal loop continues while the stack is not empty.

4.1.5 `clear()`

The `clear()` function removes all elements from a container. Before each simulation, the graph's vectors are cleared so that the next randomly generated network does not retain data from a previous run.

4.2 Existing Algorithm Implementation

The existing comparison logic contains two ideas. First, the thesis discusses MST-based routing, where a weighted tree connects the vertices while avoiding cycles. Second, the source code implements a modified DSDV-style routing-table update for comparison with the DFS reachability result.

Each node acts as a router and maintains costs to other nodes. Repeated routing-table updates propagate cost information. In the switching-enabled case, the comparison permits a route regardless of a fixed channel identifier. In the no-switching case, an update is permitted only when the relevant link supports the selected channel.

These procedures are retained as comparison models. They are not part of the proposed DFS topology-formation algorithm.

4.3 Proposed DFS Algorithm Implementation

Each network node stores a neighbor list, channel lists indexed by neighbor, a distance list, and a channel-connectivity matrix. A separate visited array records whether a node has already been added to the DFS tree.

The `solve()` function accepts a source, a destination, a channel identifier, and a switching flag. It initializes a stack with feasible outgoing edges from the source. The algorithm repeatedly pops an edge, adds it to the tree if the destination node has not been visited, and pushes feasible outgoing edges from that node.

> **IMPLEMENTATION EVIDENCE**
>
> The source code declares `stack<node2> myStack`, uses `top()` and `pop()`, and is explicitly commented as *Depth-First Search with a stack*. This confirms that the proposed implementation is DFS-based.

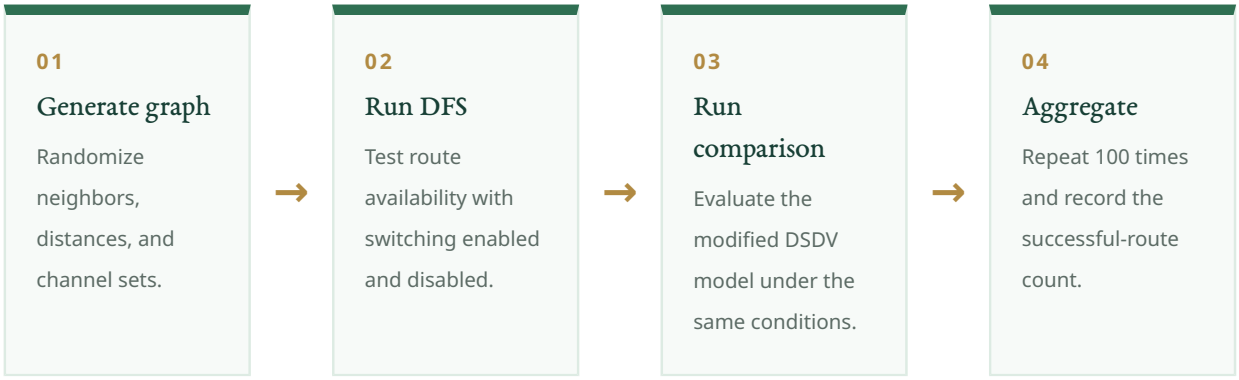
If channel switching is enabled, the algorithm accepts the available channel associated with each outgoing link. If switching is disabled, it pushes a link only when the link supports the specified channel. The function returns whether the destination was reachable.

4.4 Simulation Setup

The simulation is repeated 100 times for each configuration. Two evaluation conditions are described:

1. The number of nodes is fixed while the number of available channels varies.
2. The number of channels is fixed while the number of nodes varies.

For each generated graph, the simulator evaluates DFS reachability with switching and without switching. It also evaluates the modified DSDV comparison under the same two channel conditions. The reported statistic is the number or percentage of simulations in which a route exists between the selected source and destination.



CHAPTER

05

Experimental Results and Evaluation

The evaluation focuses on route success, the effect of channel switching, and the computational role of DFS relative to existing comparison methods.

> *The numerical claims in this chapter are preserved from the source thesis; no new experimental series has been invented.*

The multi-edge graph model is evaluated through repeated random simulations. The principal outcome is whether the source can reach the destination under a selected channel condition. Results are considered separately for switching-enabled and no-switching operation.

EVALUATION ELEMENT	DESCRIPTION
Simulator	Custom C++ graph simulator.
Runs per configuration	100 simulations.
Graph variables	Number of nodes, number of channels, randomly generated neighbor relationships, and channel sets.
Primary metric	Successful route discovery from the first node to the final node.
Proposed method	DFS-based route discovery with channel assignment.
Comparison methods	MST-based discussion and modified DSDV implementation.
Channel conditions	Switching allowed and switching not allowed.

Table 5.1. Simulation and evaluation design.

5.1 Experimental Results of the Existing Algorithm

In a multi-radio network, incoming and outgoing channels at an intermediate node do not have to be identical. The existing comparison allocates channels according to whether a channel is already occupied and whether switching is permitted.

The source results report a route-success level of approximately 95% for the described multi-radio case. This value is preserved as an approximate narrative result because the original PDF does not provide the complete underlying data series.

5.2 Simulation Study of the Proposed Model

The proposed multi-edge graph is simulated with different numbers of nodes and channels. The channels associated with each node are selected randomly. For each graph, DFS tests connectivity from the first node to the last node, and the percentage of successful routes is calculated across 100 runs.

When switching is allowed, a route can use different channels on successive links. When switching is not allowed, every selected link must support the chosen channel. The switching-enabled condition is therefore expected to retain more feasible routes, particularly as the graph grows and channel sets become heterogeneous.

The experiment evaluates reachability and route success. Because the proposed traversal is DFS, it does not establish that the discovered route is the minimum-cost or shortest route. Any such optimization would require an additional cost-aware selection mechanism.

5.3 Experimental Results of the Existing Model

The existing model uses MST-based routing for route discovery and examines both single-radio and multi-radio conditions. The source thesis reports nearly 100% route success when switching is allowed. Without switching, the success rate decreases as the number of nodes increases.

This behavior is consistent with the channel constraint: a larger route contains more links that must support a common channel when switching is disabled. Allowing per-hop channel changes removes that end-to-end common-channel requirement.

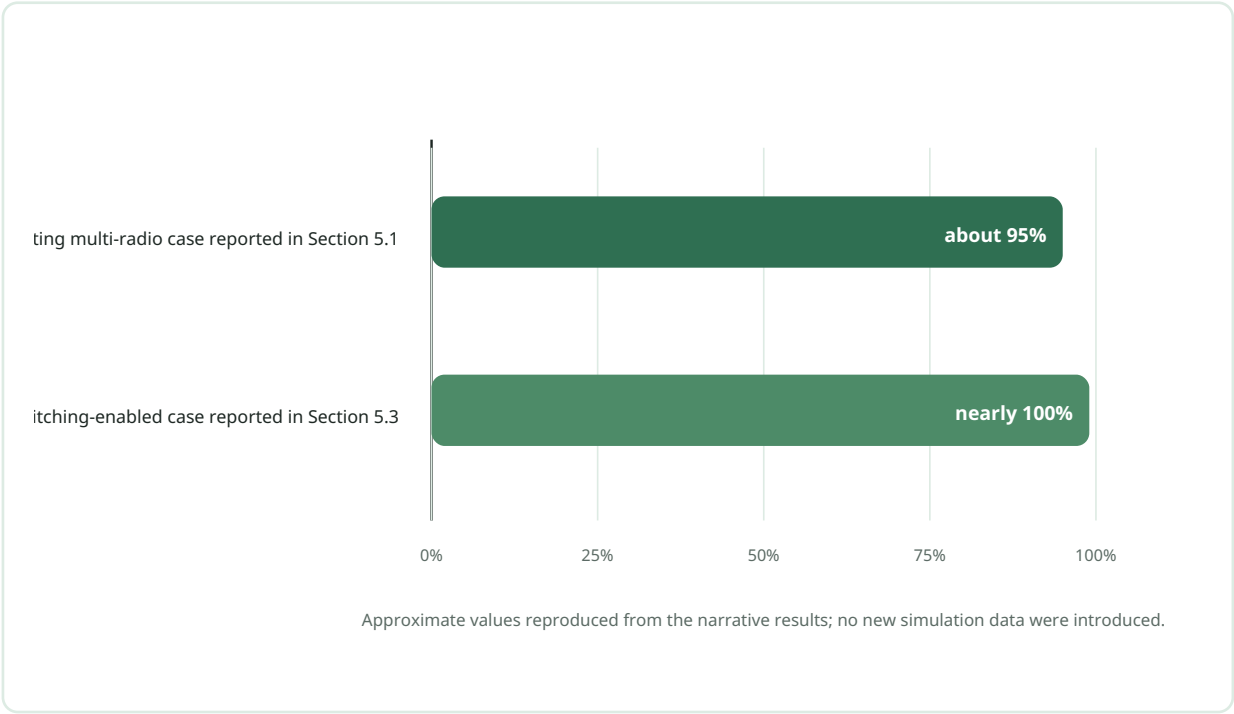


Figure 5.1. *Approximate routing-success levels stated in the original narrative results, displayed as left-to-right horizontal bars.*

5.4 Comparison of Existing and Proposed Models

The proposed DFS traversal processes each reachable node and edge in linear graph time, giving $O(V + E)$ traversal complexity. The source thesis reports the existing comparison model with complexity $O(n(E \log n)C)$, where n is the number of nodes, E the number of edges, and C the number of channels.

These expressions should be interpreted in the context of the implemented models rather than as universal complexity results for every possible MST or DSDV implementation. The principal methodological distinction is that DFS uses a stack to form a traversal tree, MST seeks a minimum-total-weight tree, and DSDV maintains routing-table information.

METHOD	ROLE	CORE MECHANISM	WHAT IT GUARANTEES IN THIS THESIS	COMPLEXITY STATEMENT
DFS	Proposed model	Stack-based graph traversal	Reachability and a loop-free DFS tree; not a shortest-path guarantee.	O(V + E) for traversal.
MST	Existing comparison	Minimum-total-weight spanning tree	A weighted tree connecting the graph when applicable.	Existing-model expression reported in the source comparison.
Modified DSDV	Existing comparison	Repeated routing-table updates	Comparison of route availability with and without channel switching.	Not isolated as a separate closed-form expression in the source thesis.

Table 5.2. Methodological roles of DFS, MST, and modified DSDV.

The comparison supports two conclusions. First, multi-edge modeling is useful because it preserves channel alternatives that a single-edge graph cannot show. Second, channel switching improves connectivity, but introduces interface and coordination overhead. The preferred configuration therefore depends on whether the design objective prioritizes route availability, minimum cost, delay, or implementation simplicity.

5.5 Chapter Summary

Real networks can contain heterogeneous nodes: some have a single radio, while others have multiple interfaces. The practical route-success rate is therefore likely to fall between the idealized no-switching and fully switching-enabled cases. The reported experiments show that switching substantially improves route availability. The proposed DFS model offers a simple linear-time traversal, while MST and modified DSDV provide distinct comparison perspectives.

CHAPTER

06

Conclusion and Future Recommendations

The thesis concludes that a multi-edge graph can express channel-dependent connectivity clearly, while DFS provides a straightforward mechanism for route discovery.

> *Future work should add cost-aware optimization, stronger spectrum prediction, and security without obscuring the model's graph structure.*

6.1 Conclusion

This thesis presented a multi-edge graph model for multi-hop, multi-channel cognitive radio networks. Parallel edges preserve the possibility that the same pair of nodes can communicate through several channels.

A C++ simulator was used to generate networks, assign channel sets, and evaluate route feasibility. The proposed routing procedure uses Depth-First Search with an explicit stack. DFS forms a traversal tree and tests whether the destination is reachable under the selected switching condition. MST and modified DSDV are retained only as existing or comparison methods.

The study shows that channel switching can substantially increase route availability because adjacent hops are no longer required to share a single end-to-end channel. The DFS traversal has $O(V + E)$ time complexity. However, DFS alone does not guarantee a shortest or minimum-cost route; its value in the present model is simplicity, reachability testing, and loop-free tree formation.

Overall, the multi-edge representation provides a direct and understandable bridge between cognitive-radio channel availability and conventional graph traversal.

6.2 Future Improvements

The model can be extended in several directions:

- Cost-aware routing: combine DFS-based reachability with Dijkstra, A*, or another optimization procedure when a minimum-cost path is required.
- Learning-based adaptation: use machine learning or reinforcement learning to predict channel availability and adapt routing decisions.
- Richer edge weights: incorporate interference, energy consumption, delay, bandwidth, link reliability, and switching cost.
- Primary-user protection: improve sensing and policy mechanisms so that secondary communication remains non-disruptive.
- Security: address malicious nodes, falsified sensing reports, jamming, and routing attacks.
- Reproducible experiments: use deterministic seeds, publish complete data series, and compare route success, delay, throughput, and packet-delivery ratio across multiple baselines.

These extensions would strengthen the model while preserving its central idea: a channel-aware graph should represent the dynamic choices available to a cognitive radio network.

Bibliography

1. M. L. Sichitiu, "Wireless mesh networks: opportunities and challenges," in *Proceedings of the World Wireless Congress*, vol. 2, p. 21, 2005.

2. I. F. Akyildiz, X. Wang, and W. Wang, "Wireless mesh networks: a survey," *Computer Networks*, vol. 47, no. 4, pp. 445-487, 2005.

3. G. P. Joshi, S. Y. Nam, and S. W. Kim, "Cognitive radio wireless sensor networks: applications, challenges and research trends," *Sensors*, vol. 13, no. 9, pp. 11196-11228, 2013.

4. Q. Zhao and A. Swami, "A survey of dynamic spectrum access: signal processing and networking perspectives," Department of Electrical and Computer Engineering, University of California, Davis, Technical Report, 2007.

5. E. Ahmed, M. Shiraz, and A. Gani, "Spectrum-aware distributed channel assignment for cognitive radio wireless mesh networks," *Malaysian Journal of Computer Science*, vol. 26, no. 3, pp. 232-250, 2013.

6. N. Mansoor, A. K. M. Muzahidul Islam, M. Zareei, S. Baharun, T. Wakabayashi, and S. Komaki, "Cognitive radio ad-hoc network architectures: a survey," *Wireless Personal Communications*, vol. 81, no. 3, pp. 1117-1142, 2015.

7. Y. R. Kondareddy and P. Agrawal, "A graph based routing algorithm for multi-hop cognitive radio networks," in *Proceedings of the 4th Annual International Conference on Wireless Internet*, p. 63, 2008.

8. I. F. Akyildiz, W.-Y. Lee, M. C. Vuran, and S. Mohanty, "A survey on spectrum management in cognitive radio networks," *IEEE Communications Magazine*, vol. 46, no. 4, pp. 40-48, 2008.

9. C. Hernandez, C. Salgado, H. Lopez, and E. Rodriguez-Colina, "Multivariable algorithm for dynamic channel selection in cognitive radio networks," *EURASIP Journal on Wireless Communications and Networking*, no. 216, 2015.

10. A. Rabbachin, T. Q. S. Quek, and M. Z. Win, "Statistical modeling of cognitive network interference," in *2010 IEEE Global Telecommunications Conference (GLOBECOM)*, pp. 1-6, 2010.

11. Y. Saleem, A. Bashir, E. Ahmed, J. Qadir, and A. Baig, "Spectrum-aware dynamic channel assignment in cognitive radio networks," in *2012 International Conference on Emerging Technologies*, pp. 1-6, 2012.

12. A. P. Subramanian, H. Gupta, S. R. Das, and J. Cao, "Minimum interference channel assignment in multi-radio wireless mesh networks," *IEEE Transactions on Mobile Computing*, vol. 7, no. 12, pp. 1459-1473, 2008.

13. A. H. Mohsenian-Rad and V. W. S. Wong, "Joint optimal channel assignment and congestion control for multi-channel wireless mesh networks," in *2006 IEEE International Conference on Communications*, vol. 5, pp. 1984-1989, 2006.

14. V. Kapse and U. Shrawankar, "Interference-aware channel assignment for maximizing throughput in WMN," *arXiv preprint arXiv:1305.2838*, 2013.

15. A. Singh and K. Singh, "Load balancing based channel assignment in multicast wireless mesh network," pp. 865-869, Mar. 2014.

16. S. Sultana and A. Asaduzzaman, "A minimum spanning tree based routing protocol for multi-hop and multi-channel cognitive radio," pp. 206-211, Jun. 2018.

Appendix

Source Code for DFS-Based Routing and Modified DSDV Comparison

The source code has been re-typeset for readability. Indentation and comments were normalized, but the computational logic was preserved. The appendix title and comments now identify DFS consistently.

```
#include <bits/stdc++.h>
using namespace std;

const int inf = 1e9 + 7, SIZE = 105;

int solve(int source, int destination, int channelID, int isSwithingAllowd);
void modifiedDSDV(int channelID, int nodeNumber, int isSwithingAllowed);
void generateGraph(int totalNode, int channel);
void simulateMySystem(void);
void clearAll(void);
void clearRoutingTable(int nodeNumber, int isSwithingAllowed);
void printGraph(int nodeNumber);

struct node {
    vector<int> neighbourList, channelList[SIZE], distance;
    int hasConnectionWith[SIZE][12];
} Network[SIZE];

struct node2 {
    int from, to, channel, cost;

    node2(int from, int to, int channel, int cost) {
        this->from = from;
        this->to = to;
        this->channel = channel;
        this->cost = cost;
    }

    bool operator<(const node2 &p) const {
        return p.cost < cost;
    }
};

struct road {
    int from, to, channel;

    road(int from, int to, int channel) {
        this->from = from;
        this->to = to;
        this->channel = channel;
    }
};

vector<road> Graph;
unordered_set<int> channelsOfSource;
int totalNode, totalCost, finalCost;
int routingTable[101][101][11];
int isConnected[SIZE];
int hasRoad[SIZE][SIZE];

void generateGraph(int totalNode, int channel) {
    int n, i, j, k, neighbourNo, c, d, temp, source, destination;
    memset(hasRoad, 0, sizeof(hasRoad));

    for (i = 1; i <= totalNode; i++) {
        if (totalNode > 2)
            neighbourNo = 1 + rand() % min(3, totalNode - 1);
        else
            neighbourNo = 1;

        set<int> neighbourSet;

        for (j = 1; j <= totalNode; j++) {
            if (hasRoad[i][j] == 1) {
                neighbourNo--;
```

```
                neighbourNo = max(0, neighbourNo);
            }
        }

        while (neighbourSet.size() < static_cast<size_t>(neighbourNo)) {
            n = 1 + rand() % totalNode;
            if (hasRoad[i][n] == 0 && n != i) {
                neighbourSet.insert(n);
                hasRoad[i][n] = hasRoad[n][i] = 1;
            }
        }

        set<int>::iterator itt;
        for (itt = neighbourSet.begin(); itt != neighbourSet.end(); ++itt) {
            d = 1 + rand() % 1007;
            Network[i].neighbourList.push_back(*itt);
            Network[*itt].neighbourList.push_back(i);
            Network[i].distance.push_back(d);
            Network[*itt].distance.push_back(d);

            set<int> channelSet;
            int channelNumber = 1 + rand() % channel;
            while (channelSet.size() != static_cast<size_t>(channelNumber)) {
                channelSet.insert(1 + rand() % 10);
            }

            set<int>::iterator itt2;
            for (itt2 = channelSet.begin(); itt2 != channelSet.end(); ++itt2) {
                Network[i].channelList[*itt].push_back(*itt2);
                Network[*itt].channelList[i].push_back(*itt2);
                Network[i].hasConnectionWith[*itt][*itt2] = 1;
                Network[*itt].hasConnectionWith[i][*itt2] = 1;
            }
        }
    }
}

void simulateMySystem(void) {
    int i, j, switching, withSwithcing, withoutSwitching, n;
    int dsdvWithoutSwitching, dsdvWithSwitching;
    int nodeNumber = 10;

    for (int channelNumber = 2; channelNumber <= 10; channelNumber++) {
        withSwithcing = 0;
        withoutSwitching = 0;
        dsdvWithoutSwitching = 0;
        dsdvWithSwitching = 0;

        for (int numberOfSimulation = 1;
             numberOfSimulation <= 100;
             numberOfSimulation++) {
            clearAll();
            generateGraph(nodeNumber, channelNumber);
            n = Network[1].neighbourList[0];
            withSwithcing += solve(
                1,
                nodeNumber,
                Network[1].channelList[n][0],
                1
            );

            channelsOfSource.clear();
            for (i = 0; i < static_cast<int>(Network[1].neighbourList.size()); i++) {
                n = Network[1].neighbourList[i];
                for (j = 0;
                     j < static_cast<int>(Network[1].channelList[n].size());
                     j++) {
                    channelsOfSource.insert(Network[1].channelList[n][j]);
                }
            }

            unordered_set<int>::iterator itt;
            for (itt = channelsOfSource.begin();
                 itt != channelsOfSource.end();
                 ++itt) {
                int isPossible = solve(1, nodeNumber, *itt, 0);
                if (isPossible == 1) {
                    withoutSwitching++;
                    break;
                }
            }

            // DFS-based routing block ends.
            // Modified DSDV comparison block begins.
            clearRoutingTable(nodeNumber, 1);
            modifiedDSDV(1, nodeNumber, 1);  // With switching.
            if (routingTable[1][nodeNumber][1] != inf)
```

```
                dsdvWithSwitching++;

            for (int i = 1; i <= 10; i++) {
                clearRoutingTable(nodeNumber, 0);
                modifiedDSDV(i, nodeNumber, 0);
                if (routingTable[1][nodeNumber][i] != inf) {
                    dsdvWithoutSwitching++;
                    break;
                }
            }
            // Modified DSDV comparison block ends.
        }

        printf(
            "For %d nodes and %d channel: %d %d | %d %d\n",
            nodeNumber,
            channelNumber,
            withoutSwitching,
            withSwithcing,
            dsdvWithoutSwitching,
            dsdvWithSwitching
        );
    }
}

void modifiedDSDV(int cID, int nodeNumber, int isSwithingAllowed) {
    int to, cost, chnl;

    for (int itteration = 1; itteration <= nodeNumber; itteration++) {
        for (int i = 1; i <= nodeNumber; i++) {
            for (int j = 0;
                 j < static_cast<int>(Network[i].neighbourList.size());
                 j++) {
                to = Network[i].neighbourList[j];
                cost = Network[i].distance[j];

                for (int k = 1; k <= nodeNumber; k++) {
                    if (isSwithingAllowed == 1) {
                        routingTable[i][k][1] = min(
                            routingTable[i][k][1],
                            routingTable[i][to][1] + routingTable[to][k][1]
                        );
                    } else if (Network[i].hasConnectionWith[to][cID] == 1) {
                        routingTable[i][k][cID] = min(
                            routingTable[i][k][cID],
                            routingTable[i][to][cID] + routingTable[to][k][cID]
                        );
                    }
                }
            }
        }
    }
}

void printGraph(int nodeNumber) {
    for (int i = 1; i <= nodeNumber; i++) {
        for (int j = 0;
             j < static_cast<int>(Network[i].neighbourList.size());
             j++) {
            printf(
                "%d -> %d %d c =",
                i,
                Network[i].neighbourList[j],
                Network[i].distance[j]
            );

            for (int k = 0;
                 k < static_cast<int>(
                     Network[i]
                         .channelList[Network[i].neighbourList[j]]
                         .size()
                 );
                 k++) {
                printf(
                    " %d",
                    Network[i]
                        .channelList[Network[i].neighbourList[j]][k]
                );
            }
            printf("\n");
        }
    }
}

void clearAll(void) {
    for (int i = 1; i <= 101; i++) {
        Network[i].neighbourList.clear();
```

```
        Network[i].distance.clear();
        memset(
            Network[i].hasConnectionWith,
            0,
            sizeof(Network[i].hasConnectionWith)
        );

        for (int j = 0; j < 104; j++) {
            Network[i].channelList[j].clear();
        }
    }
}

void clearRoutingTable(int nodeNumber, int isSwithingAllowed) {
    for (int i = 1; i <= nodeNumber; i++) {
        for (int j = i; j <= nodeNumber; j++) {
            for (int c = 0; c <= 10; c++) {
                if (i == j) {
                    routingTable[i][i][c] = 0;
                } else {
                    routingTable[i][j][c] = inf;
                    routingTable[j][i][c] = inf;
                }
            }
        }
    }

    if (isSwithingAllowed == 1) {
        for (int i = 1; i <= nodeNumber; i++) {
            for (int j = 0;
                 j < static_cast<int>(Network[i].neighbourList.size());
                 j++) {
                int n = Network[i].neighbourList[j];
                for (int c = 0; c <= 10; c++) {
                    routingTable[i][n][c] = Network[i].distance[j];
                }
            }
        }
    } else {
        for (int i = 1; i <= nodeNumber; i++) {
            for (int j = 0;
                 j < static_cast<int>(Network[i].neighbourList.size());
                 j++) {
                int n = Network[i].neighbourList[j];
                for (int c = 0;
                     c < static_cast<int>(Network[i].channelList[n].size());
                     c++) {
                    int ch = Network[i].channelList[n][c];
                    routingTable[i][n][ch] = Network[i].distance[j];
                    routingTable[n][i][ch] = Network[i].distance[j];
                }
            }
        }
    }
}

int solve(
    int source,
    int destination,
    int channelID,
    int isSwithingAllowed
) {
    /*
     * Depth-First Search with a stack.
     */
    int isPossible = 0;
    memset(isConnected, 0, sizeof(isConnected));
    isConnected[source] = 1;
    Graph.clear();
    stack<node2> myStack;

    for (int i = 0;
         i < static_cast<int>(Network[source].neighbourList.size());
         i++) {
        int to = Network[source].neighbourList[i];
        int cost = Network[source].distance[i];
        int chnl = Network[source].channelList[to][0];

        if (isSwithingAllowed == 1) {
            myStack.push(node2(source, to, chnl, cost));
        } else if (Network[source].hasConnectionWith[to][channelID] == 1) {
            myStack.push(node2(source, to, channelID, cost));
        }
    }

    while (myStack.empty() != true) {
        node2 temp = myStack.top();
```

```
            myStack.pop();

            if (isConnected[temp.to] == 0) {
                Graph.push_back(road(temp.from, temp.to, temp.channel));
                if (temp.to == destination)
                    isPossible = 1;

                isConnected[temp.to] = 1;
                totalCost += temp.cost;

                for (int i = 0;
                     i < static_cast<int>(Network[temp.to].neighbourList.size());
                     i++) {
                    int to = Network[temp.to].neighbourList[i];
                    int cost = Network[temp.to].distance[i];
                    int chnl = Network[temp.to].channelList[to][0];

                    if (isSwithingAllowed == 1) {
                        myStack.push(node2(temp.to, to, chnl, cost));
                    } else if (
                        Network[temp.to].hasConnectionWith[to][channelID] == 1
                    ) {
                        myStack.push(node2(temp.to, to, channelID, cost));
                    }
                }
            }
        }

        if (isPossible == 0)
            totalCost = 9999999;

        return isPossible;
    }

    int main() {
        srand(time(NULL));
        simulateMySystem();
        return 0;
    }
```